RAINBOW: VIOLET THREAD, ORTHOGONAL-CUBIC PROJECTIVE GEOMETRY, REFLECTION, PÓLYA, AND PLUECKER REDUCTIONS

VÍCTOR CASAÑA-GINER

Abstract. For $1/2 \le \beta \le 1$, the Rainbow family is $W_\beta(t)$, with Violet exactly the fibre $\beta = 1/2$. The primary geometric object is therefore retained at the start of the paper:

$$W(t) = W_{1/2}(t) = Z(t)e^{-i\vartheta(t)}, \qquad \Gamma(t) = (t, \Re W(t), \Im W(t)).$$

Its contacts with the value-plane origin are the Violet nodes $\Gamma(\gamma_n) = (\gamma_n, 0, 0)$, and consecutive nodes delimit the three-dimensional Violet lobes. Each lobe supplies one fixed phase coordinate q, and that coordinate is carried unchanged through the complete Rainbow family; no node is moved or searched as β varies.

The orthogonal-cubic quotient $Q_\perp = I - \Pi_3$ produces the projected lobe state $u_{\beta,n} = Q_\perp W_{\beta,n}$ and its projective defect $M_n(\beta)$. The dense differential audit records 64,560/64,560 nonnegative sampled derivatives and 240/240 complete lobe paths for this quotient, while the corresponding sign law is false for several other fixed quotients. A separate direct scan of the mixed curvature $\mathcal{C}(\alpha, \beta) = \partial_{\alpha\beta} \log |K(\alpha, \beta)|$ found no negative sampled values on the reported grids for lobes 2, 8, and 25.

The analytic content is deliberately restricted to unconditional identities, negative results, and explicit open targets. The Rainbow flow has the fixed generator $L_n = -ia_n(q)\partial_q$. The projected Gram kernel gives an exact formula for M' and an exact Pluecker representation of the mixed curvature. The raw modal generator is structurally purely imaginary in every real modal basis, and the constant-coefficient Legendre model admits no fixed diagonal phase gauge making all nonzero couplings cooperative; those routes are closed. The coherence matrix $R = cc^*$ gives a closed lift of the modal dynamics. Reflection-conjugation splits the state and the generator into exact symmetric and antisymmetric components and reduces the modal phases to two real sequences. The Pluecker vector admits both an exact cosine decomposition and an accumulated projective-torsion law. In parallel, the Pólya calculation reduces monotone real-negative root flow of the finite modal generating polynomial to the corrected adjacent modal-ratio sign.

No analytic theorem asserting nonvanishing of $u_\beta \wedge u'_\beta$, TP2 on a parameter band, or positivity of $M'_n(\beta)$ on a nontrivial interval is claimed here. The exact diagonal identity $\mathcal{C}(\beta, \beta) = P(\beta) \ge 0$ holds on the regular projective set, and the remaining global problem is stated directly as the sign of the integrated mixed curvature. Standard nondegeneracy of the classical Pluecker pairing or Grassmannian does not settle the possible vanishing of the specific Rainbow bivector $u_\beta \wedge u'_\beta$ and is not used as a substitute.

Contents

1. Rainbow and the Violet fibre 2
1.1. Violet nodes and zero-to-zero lobes 3
1.2. Rainbow lobe fibres 4
2. Fixed Rainbow transverse generator 4
3. Scope and logical status 6
4. The orthogonal-cubic quotient 6
5. Finite quotient and differential audits 6
6. Projected Gram kernel and exact mixed curvature 7
7. Supplementary direct pointwise-curvature scan 9

Date: Version 5 — August 2026.

2020 *Mathematics Subject Classification.* 11M26, 46C05, 15B48, 30B40, 26C10.

Key words and phrases. Rainbow family, Violet fibre, Violet Thread, zero-to-zero lobes, projective Hilbert geometry, orthogonal quotient, total positivity, TP2 kernel, Pólya frequency sequence, reflection conjugation.

ORCID: 0000-0002-8252-2608.

8. Diagonal curvature and exact far-memory decomposition 9
9. Why the naive cooperative-generator route fails 9
9.1. Formal self-adjoint/rotational split 10
10. Modal equations and the corrected Chebyshev sign 10
11. The coherence matrix: a closed lift of the modal dynamics 11
12. Reflection-conjugation reduction 12
12.1. Canonical choice of the reflection phase 12
13. Exact modal phase structure of the reflection split 13
14. Plücker decomposition and perturbative aguteness 13
14.1. Modal Pluecker identity and accumulated torsion 14
14.2. Projective stationarity and the genuine nonvanishing problem 15
15. Internal bound on the antisymmetric forcing 16
16. Pólya–Chebyshev module 17
16.1. Exact algebraic root-flow reduction 17
16.2. Hermitian form for the root speed 17
17. The phase bridge: modulus kernel and coherence factor 18
18. Current analytic status and surviving routes 18
19. Bibliographic position 19
20. Reproducibility package 19
References 19
Appendix A. MATLAB: five-quotient rigidity audit 20
Appendix B. MATLAB: dense differential projective-flow audit 27
Appendix C. MATLAB: pointwise curvature, reflection, and Pólya diagnostics 32

1. Rainbow and the Violet fibre

Definition 1.1 (Rainbow family and Violet fibre)**.** For

$$\frac{1}{2} \le \beta \le 1, \tag{1.1}$$

define the Rainbow functions

$$\boxed{W_\beta(t) = \zeta(\beta + it).} \tag{1.2}$$

The complete family $\{W_\beta\}_{1/2\le\beta\le1}$ is called *Rainbow*. Its distinguished fibre

$$\boxed{W(t) := W_{1/2}(t) = \zeta\Big(\frac{1}{2} + it\Big) = u(t) + iv(t)} \tag{1.3}$$

is called *Violet*.

The central three-dimensional Violet trace is

$$\boxed{\Gamma(t) = \big(t, u(t), v(t)\big).} \tag{1.4}$$

The classical Hardy decomposition [1] is

$$\boxed{W(t) = Z(t)e^{-i\vartheta(t)}, \qquad Z(t) \in \mathbb{R}.} \tag{1.5}$$

Consequently

$$\Gamma(t) = \big(t, Z(t)\cos\vartheta(t), -Z(t)\sin\vartheta(t)\big). \tag{1.6}$$

Differentiation gives

$$W_t(t) = e^{-i\vartheta(t)}\big(Z'(t) - i\vartheta'(t)Z(t)\big) \tag{1.7}$$

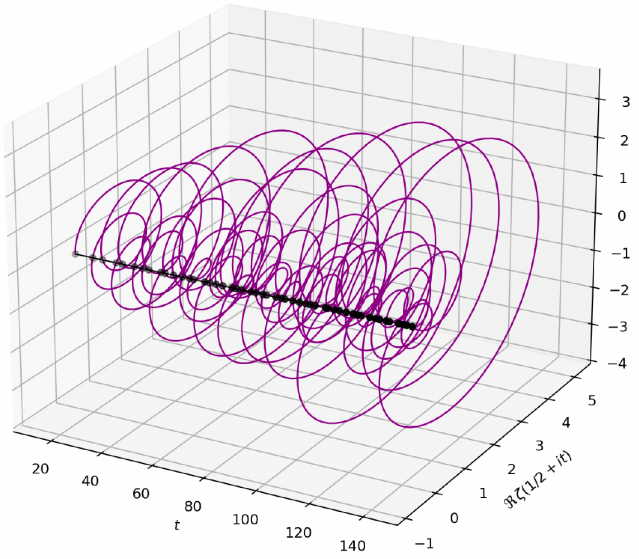


FIGURE 1. The Euclidean Violet Thread over the first fifty carried node ordinates. The black points are the contacts $\Gamma(\gamma_n) = (\gamma_n, 0, 0)$; each violet arc between consecutive contacts is one genuine three-dimensional lobe.

and hence

$$|W_t(t)|^2 = Z'(t)^2 + \vartheta'(t)^2 Z(t)^2. \tag{1.8}$$

1.1. **Violet nodes and zero-to-zero lobes.** Let $\gamma_n < \gamma_{n+1}$ be consecutive contacts of the Violet trace with the value-plane origin:

$$\boxed{W(\gamma_n) = 0, \qquad \Gamma(\gamma_n) = (\gamma_n, 0, 0).} \tag{1.9}$$

The nth Violet lobe is the curve segment

$$\boxed{\mathcal{L}_n = \{\Gamma(t) : \gamma_n \le t \le \gamma_{n+1}\}.} \tag{1.10}$$

On

$$I_n = (\gamma_n, \gamma_{n+1}) \tag{1.11}$$

the sign of Z is constant. Put

$$\varepsilon_n = \operatorname{sgn} Z(t), \qquad t \in I_n, \tag{1.12}$$

and define the fixed phase sweep

$$\Omega_n = \vartheta(\gamma_{n+1}) - \vartheta(\gamma_n). \tag{1.13}$$

Rotate only by the fixed Violet lobe phase:

$$\widehat{W}_{V,n}(t) = \varepsilon_n e^{i\vartheta(\gamma_n)} W(t) = |Z(t)| e^{-i[\vartheta(t) - \vartheta(\gamma_n)]}. \tag{1.14}$$

Introduce the uniform lobe phase coordinate

$$q = \frac{\vartheta(t) - \vartheta(\gamma_n)}{\Omega_n}, \qquad 0 \le q \le 1, \tag{1.15}$$

or equivalently define $t_n(q)$ by

$$\boxed{\vartheta(t_n(q)) = \vartheta(\gamma_n) + \Omega_n q.} \tag{1.16}$$

With

$$r_n(q) = |Z(t_n(q))|, \tag{1.17}$$

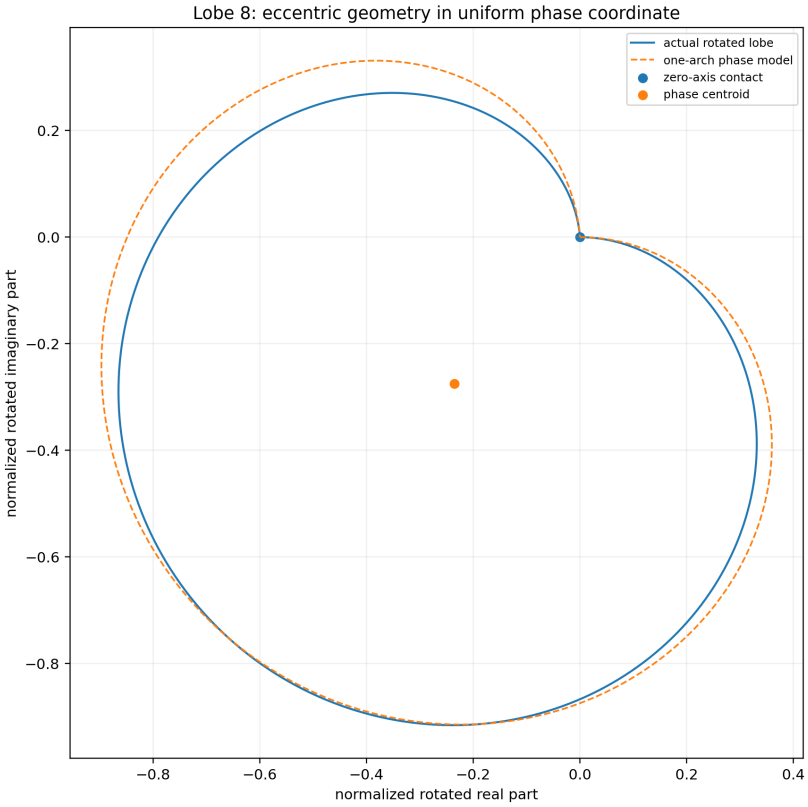


FIGURE 2. A representative Violet lobe in the uniform phase coordinate. The two endpoints are the consecutive Violet nodes; the fixed map $q \mapsto t_n(q)$ is the coordinate carried into every Rainbow fibre.

the rotated Violet lobe is exactly

$$\boxed{\widehat{W}_{V,n}(q) = r_n(q)e^{-i\Omega_n q}, \qquad r_n(0) = r_n(1) = 0.} \tag{1.18}$$

Thus the node–lobe–node geometry is retained before any quotient or projective construction is introduced.

1.2. **Rainbow lobe fibres.** The Rainbow deformation does not move the Violet nodes. Instead, each Violet interval and its phase coordinate are frozen and used to sample every fibre:

$$\boxed{W_{\beta,n}(q) = W_\beta(t_n(q)), \qquad \frac{1}{2} \le \beta \le 1.} \tag{1.19}$$

At $\beta = 1/2$ this is exactly the original Violet lobe before the fixed phase rotation. For $\beta > 1/2$, the values at $q = 0, 1$ are not declared to be nodes; the endpoints remain the fixed Violet coordinates γ_n, γ_{n+1}.

The same fixed rotation may be applied throughout the family,

$$\widehat{W}_{\beta,n}(q) = \varepsilon_n e^{i\vartheta(\gamma_n)} W_{\beta,n}(q), \tag{1.20}$$

and is projectively immaterial because the multiplier is independent of β and q. In the remainder of the paper $W_{\beta,n}$ denotes the unrotated Rainbow fibre unless a rotated lobe is explicitly required.

2. Fixed Rainbow transverse generator

Put

$$F(\delta, t) = W_{1/2+\delta}(t), \qquad \delta = \beta - \frac{1}{2}. \tag{2.1}$$

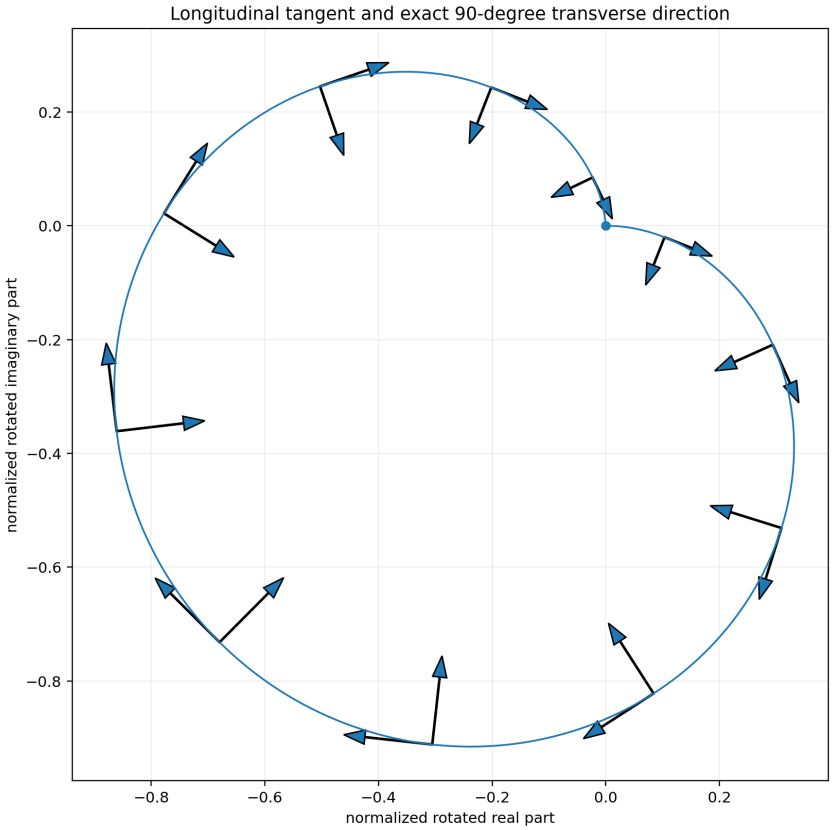


FIGURE 3. Exact transverse quarter-turn field on a Violet lobe. This is the geometric origin of the fixed Rainbow generator $L_n = -ia_n(q)\partial_q$.

Analyticity gives the exact quarter-turn identity

$$\boxed{F_\delta = -iF_t.} \tag{2.2}$$

Differentiating (1.16) yields

$$\vartheta'(t_n(q))t_n'(q) = \Omega_n. \tag{2.3}$$

Hence every fixed Violet lobe generates the autonomous Rainbow evolution

$$\boxed{\partial_\beta W_{\beta,n} = L_n W_{\beta,n}, \qquad L_n = -ia_n(q)\partial_q, \qquad a_n(q) = \frac{\vartheta'(t_n(q))}{\Omega_n}.} \tag{2.4}$$

The coefficient a_n depends on the fixed Violet lobe coordinate but not on β.

Proposition 2.1 (Fixed-generator Rainbow evolution)**.** *For each fixed lobe,* (2.4) *is an autonomous evolution in the Rainbow parameter β. On analytic vectors it is equivalent to the exact complex translation generated by $-i\partial_t$.*

Proof. Equation (2.2) is $\partial_\delta F = (-i\partial_t)F$. Since $t_n(q)$ is fixed once the Violet lobe is chosen, the chain rule converts $-i\partial_t$ to $-ia_n(q)\partial_q$, with no residual β dependence. □

Lemma 2.2 (Internal variation of the Rainbow coefficient)**.** *One has the exact identity*

$$\boxed{a_n'(q) = \frac{\vartheta''(t_n(q))}{\vartheta'(t_n(q))}.} \tag{2.5}$$

Proof. Differentiate $a_n(q) = \vartheta'(t_n(q))/\Omega_n$ and use $t_n'(q) = \Omega_n/\vartheta'(t_n(q))$. □

TABLE 1. Dense differential audit on 240 lobes.

Quotient	Nonnegative samples	Lobes all-beta	Minimum M_n'
Hermite jet 1	64556/64560	239/240	-4.063×10^{-9}
Endpoint values	64299/64560	227/240	-8.741
Hermite jet 2	62735/64560	182/240	-9.120×10^{-7}
Minimal boundary spline	64544/64560	239/240	-1.179×10^{-7}
Orthogonal cubic	64560/64560	240/240	$+3.412 \times 10^{-10}$

3. Scope and logical status

The primary objects of this paper are the Rainbow family W_β, its Violet fibre $W = W_{1/2}$, the Violet nodes γ_n, the Violet lobes $\mathcal{L}_n$, and the fixed lobe flow (2.4). The formula (1.2) defines the family; after that definition, the proof program developed below is internal to these Rainbow objects: fixed lobe coordinates, orthogonal projection, projective geometry, modal transport, reflection, coherence matrices, Pólya root flow, and Pluecker bivectors.

No function $\gamma_n(\beta)$ is introduced. No node is transported or searched away from Violet. Numerical evaluators are used only to evaluate the already defined Rainbow fibres in the finite experiments. Exact identities, finite computations, negative results, and unresolved analytic targets are kept distinct throughout.

4. The orthogonal-cubic quotient

Let

$$\mathcal{H} = L^2([0,1],\, \mathrm{d}q; \mathbb{C}), \qquad \langle f, g\rangle = \int_0^1 \overline{f(q)} g(q)\, \mathrm{d}q. \tag{4.1}$$

Define

$$\mathcal{P}_3 = \operatorname{span}\{1, q, q^2, q^3\}, \qquad Q_\perp = I - \Pi_3, \tag{4.2}$$

where Π_3 is the orthogonal projection onto $\mathcal{P}_3$. Set

$$u_{\beta,n} = Q_\perp W_{\beta,n}, \qquad r_{\beta,n} = \frac{u_{\beta,n}}{\|u_{\beta,n}\|}. \tag{4.3}$$

For $\beta_0 = 1/2$ define the projective defect

$$\boxed{M_n(\beta) = 1 - |\langle r_{\beta_0,n}, r_{\beta,n}\rangle|^2\,.} \tag{4.4}$$

Proposition 4.1 (Canonical quotient metric)**.** *$Q_\perp f$ is the unique minimum-norm representative of the quotient class $[f] \in \mathcal{H}/\mathcal{P}_3$, and*

$$\langle [f], [g]\rangle_{\mathcal{H}/\mathcal{P}_3} := \langle Q_\perp f, Q_\perp g\rangle \tag{4.5}$$

defines the natural Hilbert inner product on the quotient.

Proof. The orthogonal decomposition $f = \Pi_3 f + Q_\perp f$ gives

$$\|Q_\perp f + p\|^2 = \|Q_\perp f\|^2 + \|p\|^2 \tag{4.6}$$

for $p \in \mathcal{P}_3$. The minimum and representative-independence follow immediately. □

5. Finite quotient and differential audits

The quotient-rigidity computation used 240 consecutive lobes, a 49-point β grid, and five fixed linear quotients. Four gave 240/240 discretely monotone lobe paths; the endpoint-only quotient gave 235/240. The intersection across all five quotients was 235/240. This establishes robustness of the ordering at the sampled scale, but not a differential law.

The denser differential audit used 270 β samples. The universal cross-quotient statement is false. The orthogonal-cubic quotient, however, was the only tested quotient with no sampled differential failure.

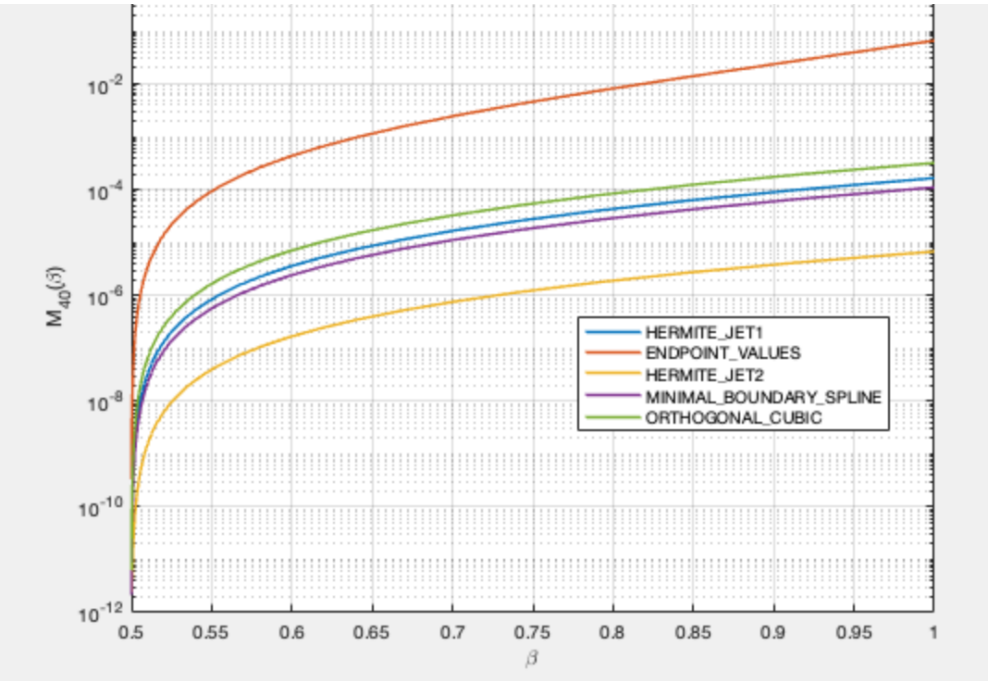


FIGURE 4. Projective defect of lobe 40 under five fixed quotient constructions. The top application chrome and plot title are clipped; the plotted data are unchanged. The quotient changes the scale substantially while preserving the sampled ordering for this lobe.

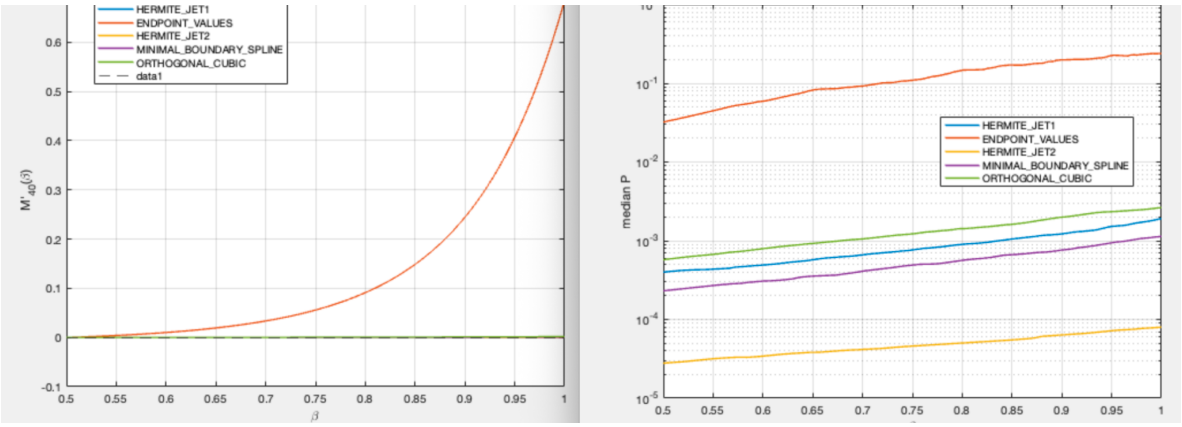


FIGURE 5. Differential lobe-40 deformation and median intrinsic projective speed from the dense audit. The original panel titles are clipped; no data are altered.

The orthogonal-cubic row motivates the sign problem

$$\boxed{M_n'(\beta) \overset{?}{\geq} 0, \qquad \frac{1}{2} \leq \beta \leq 1.} \tag{5.1}$$

The finite audit does not prove (5.1).

6. Projected Gram kernel and exact mixed curvature

For the rest of the analytic discussion, $Q = Q_\perp$ and the lobe index is suppressed. Define

$$K(\alpha, \beta) = \langle u_\alpha, u_\beta \rangle . \tag{6.1}$$

Then

$$1 - M(\beta) = \frac{|K(\beta_0, \beta)|^2}{K(\beta_0, \beta_0) K(\beta, \beta)}. \tag{6.2}$$

TABLE 2. Direct pointwise mixed-curvature scan for the orthogonal-cubic quotient. Endpoints are omitted from the displayed pair counts.

Lobe	β grid	Interior pairs	$\min \mathcal{C}$	$\max \mathcal{C}$
2	11 points	81	9.0×10^{-5}	1.1×10^{-4}
8	21 points	361	1.64×10^{-3}	2.49×10^{-3}
25	11 points	81	8.8×10^{-4}	1.50×10^{-3}

On any connected region where $K \neq 0$, define

$$\mathcal{C}(\alpha, \beta) = \partial_\alpha \partial_\beta \log |K(\alpha, \beta)|. \tag{6.3}$$

Theorem 6.1 (Exact projective derivative)**.** *One has*

$$\boxed{M'(\beta) = 2(1 - M(\beta)) \int_{\beta_0}^{\beta} \mathcal{C}(\alpha, \beta)\, \mathrm{d}\alpha.} \tag{6.4}$$

Moreover

$$\mathcal{C}(\alpha, \beta) = \mathrm{Re}\left[\frac{\left\langle u'_\alpha, u'_\beta \right\rangle}{K(\alpha, \beta)} - \frac{\langle u'_\alpha, u_\beta \rangle \left\langle u_\alpha, u'_\beta \right\rangle}{K(\alpha, \beta)^2} \right]. \tag{6.5}$$

Proof. Taking logarithms of (6.2) and differentiating gives

$$M'(\beta) = 2(1 - M) \left[\partial_\beta \log |K(\beta, \beta)|^{1/2} - \partial_\beta \log |K(\beta_0, \beta)| \right]. \tag{6.6}$$

The fundamental theorem of calculus in the first kernel variable gives (6.4). Differentiating $\log |K|$ in both variables gives (6.5). □

Define the intrinsic projective speed

$$P(\beta) = \frac{\left\| u'_\beta \right\|^2}{\|u_\beta\|^2} - \frac{|\left\langle u_\beta, u'_\beta \right\rangle|^2}{\|u_\beta\|^4}. \tag{6.7}$$

Proposition 6.2 (Exact diagonal projective identity)**.** *On the regular projective set $u_\beta \neq 0$,*

$$\boxed{\mathcal{C}(\beta, \beta) = P(\beta) \geq 0.} \tag{6.8}$$

Equivalently,

$$\boxed{\mathcal{C}(\beta, \beta) = \frac{\left\| u_\beta \wedge u'_\beta \right\|^2}{\|u_\beta\|^4}.} \tag{6.9}$$

Proof. Set $\alpha = \beta$ in (6.5) and use the Hilbert-space Lagrange identity

$$\|u \wedge v\|^2 = \|u\|^2 \|v\|^2 - |\langle u, v \rangle|^2. \tag{6.10}$$

□

The stronger pointwise target is

$$\boxed{\mathcal{C}(\alpha, \beta) \geq 0, \qquad \beta_0 \leq \alpha \leq \beta \leq 1.} \tag{6.11}$$

For $|K| > 0$, this is the differential TP2/log-supermodularity condition for $|K|$. It is stronger than the integrated condition actually equivalent to $M' \geq 0$.

7. Supplementary direct pointwise-curvature scan

The preceding differential experiment tested M' directly. A later direct calculation evaluated (6.5) itself for the orthogonal-cubic quotient using the same Rainbow fibre evaluator. The supplied development log records the following interior-pair results.

No negative value was reported in any of the 523 displayed interior pairs. This is stronger finite evidence than the direct M' scan because it targets the stronger pointwise condition (6.11). It remains a finite computation. Appendix C contains a MATLAB reconstruction of the pointwise-curvature calculation together with the later reflection and modal diagnostics.

8. Diagonal curvature and exact far-memory decomposition

The diagonal identity (6.8) is exact, but it does not by itself give an off-diagonal sign. Version 5 therefore makes no analytic TP2-band claim. The direct scan in Table 2 is retained as finite evidence only.

The projective integral can nevertheless be separated exactly at any chosen radius $r > 0$. For $\beta - r \geq \beta_0 = 1/2$, define, on a connected region where the displayed kernel logarithms are regular,

$$I^{(r)}_{\mathrm{near}}(\beta) = \int_{\beta-r}^{\beta} \mathcal{C}(\alpha,\beta)\,\mathrm{d}\alpha, \tag{8.1}$$

$$I^{(r)}_{\mathrm{far}}(\beta) = \int_{\beta_0}^{\beta-r} \mathcal{C}(\alpha,\beta)\,\mathrm{d}\alpha. \tag{8.2}$$

Then

$$\boxed{\frac{M'(\beta)}{2(1-M(\beta))} = I^{(r)}_{\mathrm{far}}(\beta) + I^{(r)}_{\mathrm{near}}(\beta).} \tag{8.3}$$

No sign is assigned analytically to either term in Version 5.

Put

$$g(\alpha,\beta) = \partial_\beta \log|K(\alpha,\beta)|. \tag{8.4}$$

Since $\mathcal{C} = \partial_\alpha g$, the far contribution collapses exactly to two boundary rates:

$$\boxed{I^{(r)}_{\mathrm{far}}(\beta) = g(\beta-r,\beta) - g(\beta_0,\beta).} \tag{8.5}$$

Thus the global problem does not require pointwise positivity on the whole parameter triangle. It requires control of the total integral in (6.4); (8.5) isolates the genuinely nonlocal part without assuming any positive near-diagonal margin.

The first transverse derivative of the mixed curvature is also explicit wherever $K \neq 0$:

$$\boxed{\partial_\alpha \mathcal{C} = \mathrm{Re}\left[\frac{K_{\alpha\alpha\beta}}{K} - \frac{2K_\alpha K_{\alpha\beta} + K_{\alpha\alpha}K_\beta}{K^2} + \frac{2K_\alpha^2 K_\beta}{K^3}\right].} \tag{8.6}$$

This formula is recorded as a quantitative tool. No bound extracted from it is promoted to a positivity theorem without an independent Rainbow proof of the missing nonvanishing and lower-margin information.

9. Why the naive cooperative-generator route fails

The fixed generator is structurally constrained. Let $\{\phi_m\}$ be any real orthonormal basis. Since $a_n(q)$ and ϕ_m are real,

$$G^{(n)}_{mk} := \langle \phi_m, L_n \phi_k \rangle = -iD^{(n)}_{mk}, \qquad D^{(n)}_{mk} \in \mathbb{R}. \tag{9.1}$$

Thus the modal matrix is purely imaginary for every lobe. If $c = x + iy$, then

$$\dot{x} = Dy, \qquad \dot{y} = -Dx \tag{9.2}$$

up to the chosen sign convention for D. The dynamics are cross-coupled and cannot be read as a real Metzler system in either x or y separately.

There is a stronger obstruction even in the constant-coefficient model.

Proposition 9.1 (No fixed cooperative phase gauge in the Legendre transport model)**.** *Let*

$$\phi_m(q) = \sqrt{2m+1}\, P_m(2q-1) \tag{9.3}$$

be the normalized shifted Legendre basis and suppose $a(q) \equiv a_0 > 0$. There is no fixed diagonal phase gauge $a_m = e^{i\omega_m} b_m$ that makes every nonzero off-diagonal entry of the modal generator real and positive.

Proof. The exact derivative identity is

$$\phi_k'(q) = 2 \sum_{\substack{m<k \\ k-m \text{ odd}}} \sqrt{(2m+1)(2k+1)}\, \phi_m(q). \tag{9.4}$$

Hence the entries $G_{4,5}$, $G_{5,6}$, $G_{6,7}$, and $G_{4,7}$ are all nonzero and have phase $-\pi/2$. Positivity after the diagonal phase gauge would require

$$\omega_5 - \omega_4 \equiv \omega_6 - \omega_5 \equiv \omega_7 - \omega_6 \equiv \frac{\pi}{2} \pmod{2\pi}. \tag{9.5}$$

The first three relations imply $\omega_7 - \omega_4 \equiv 3\pi/2$, whereas the direct $4 \to 7$ coupling requires $\omega_7 - \omega_4 \equiv \pi/2$. Contradiction. □

Therefore a proof of (5.1) cannot follow solely from a hidden Metzler structure of $Q_\perp L_n Q_\perp$.

9.1. **Formal self-adjoint/rotational split.** Boundary terms aside, the formal adjoint is

$$L_n^* = L_n - i a_n'(q). \tag{9.6}$$

Write

$$L_n = A_n + i B_n, \tag{9.7}$$

with

$$\boxed{A_n = -i\sqrt{a_n}\, \partial_q \sqrt{a_n}, \qquad B_n = \frac{1}{2} a_n'(q).} \tag{9.8}$$

These are formally self-adjoint under compatible domain conditions. Their noncommutation is explicit:

$$\boxed{[A_n, B_n] = -\frac{i}{2} a_n(q) a_n''(q)} \tag{9.9}$$

as a multiplication operator. Thus the relevant commutator is governed by $a_n a_n''$, not merely by the size of a_n'.

10. Modal equations and the corrected Chebyshev sign

Use the real shifted-Legendre basis and expand the full state

$$W_\beta = \sum_{m \geq 0} c_m(\beta) \phi_m. \tag{10.1}$$

Because the full generator is autonomous,

$$\boxed{c' = -iDc,} \tag{10.2}$$

where D is a real matrix. Splitting low and high modes,

$$c = (c_L, c_H), \qquad c_H = (c_4, c_5, \ldots), \tag{10.3}$$

recovers the projected equation

$$c_H' = -iD_{HH} c_H - iD_{HL} c_L. \tag{10.4}$$

The second term is exactly the modal form of $Q_\perp L \Pi_3 W$; it is not an independent source.

Let

$$p_m = |c_m|^2. \tag{10.5}$$

Then

$$\boxed{p'_m = 2\sum_k D_{mk}\,\mathrm{Im}(\overline{c_m}c_k).} \tag{10.6}$$

Consequently the p_m do not form a closed system; the relative modal phases enter through the coherences $\mathrm{Im}(\overline{c_m}c_k)$.

For adjacent high modes define

$$\boxed{\Delta_m(\beta) = p_m p'_{m+1} - p_{m+1}p'_m.} \tag{10.7}$$

This sign is the corrected one. Wherever $p_m p_{m+1} > 0$,

$$\boxed{\Delta_m \geq 0 \iff \frac{\mathrm{d}}{\mathrm{d}\beta}\log\frac{p_{m+1}}{p_m} \geq 0 \iff \frac{\mathrm{d}}{\mathrm{d}\beta}\log\frac{|c_{m+1}|}{|c_m|} \geq 0.} \tag{10.8}$$

The previously reversed 2×2 determinant sign is not used in this version.

11. The coherence matrix: a closed lift of the modal dynamics

The correct closed object one level above p_m is

$$\boxed{R(\beta) = c(\beta)c(\beta)^*.} \tag{11.1}$$

From (10.2),

$$\boxed{R' = -iDR + iRD^T.} \tag{11.2}$$

This equation contains the modal moduli and all phase coherences simultaneously.

Let P denote the diagonal projection onto indices $m \geq 4$ and set

$$\widehat{R}_\beta = PR_\beta P. \tag{11.3}$$

Then

$$\boxed{|K(\alpha,\beta)|^2 = \mathrm{Tr}(\widehat{R}_\alpha \widehat{R}_\beta).} \tag{11.4}$$

Thus the squared absolute kernel is itself a Gram kernel in the real Hilbert space of Hermitian Hilbert–Schmidt operators.

For

$$S(\alpha,\beta) = |K(\alpha,\beta)|^2, \tag{11.5}$$

TP2 of $|K|$ is equivalent to TP2 of S, because $S = |K|^2$ and all entries are nonnegative. The 2×2 minor has the exterior-algebra representation

$$\boxed{\det\big(S(\alpha_i,\beta_j)\big)_{i,j=1}^2 = \left\langle \widehat{R}_{\alpha_1} \underset{\mathrm{HS}}{\wedge} \widehat{R}_{\alpha_2}, \widehat{R}_{\beta_1} \underset{\mathrm{HS}}{\wedge} \widehat{R}_{\beta_2} \right\rangle_{\wedge^2\mathrm{HS}}.} \tag{11.6}$$

Hence the phase problem can be phrased as a cone problem for ordered Hilbert–Schmidt bivectors rather than as a demand for exact locking of every individual phase.

12. Reflection-conjugation reduction

Let $\mathcal{R}$ denote reflection, $(\mathcal{R}f)(q) = f(1-q)$. For a fixed real parameter χ define

$$\boxed{(J_\chi f)(q) = e^{2i\chi}\overline{f(1-q)}.} \tag{12.1}$$

J_χ is an antiunitary involution. Since $\mathcal{P}_3$ is real and invariant under $q \mapsto 1-q$,

$$\boxed{J_\chi Q_\perp = Q_\perp J_\chi.} \tag{12.2}$$

Split

$$a_+(q) = \frac{a(q) + a(1-q)}{2}, \qquad a_-(q) = \frac{a(q) - a(1-q)}{2}, \tag{12.3}$$

and

$$L_\pm = -ia_\pm(q)\partial_q. \tag{12.4}$$

Lemma 12.1 (Exact reflection parity of the generator)**.**

$$\boxed{J_\chi L_+ J_\chi = L_+, \qquad J_\chi L_- J_\chi = -L_-.} \tag{12.5}$$

Proof. For $L_a = -ia(q)\partial_q$, direct differentiation gives

$$J_\chi L_a J_\chi = -ia(1-q)\partial_q. \tag{12.6}$$

Apply this to the symmetric and antisymmetric parts of a. □

Define

$$X_\beta = \frac{W_\beta + J_\chi W_\beta}{2}, \qquad Y_\beta = \frac{W_\beta - J_\chi W_\beta}{2}. \tag{12.7}$$

Then $J_\chi X = X$ and $J_\chi Y = -Y$.

Proposition 12.2 (Exact coupled reflection system)**.** *The transverse evolution is equivalent to*

$$\boxed{\begin{aligned} X'_\beta &= L_+ X_\beta + L_- Y_\beta, \\ Y'_\beta &= L_+ Y_\beta + L_- X_\beta. \end{aligned}} \tag{12.8}$$

Thus departure from the J_χ-fixed subspace has exactly two sources: the initial defect Y_{β_0} and the antisymmetric forcing $L_- X$.

Proof. Use $W = X + Y$, $J_\chi W = X - Y$, and (12.5) in the decomposition of $W' = LW$ into its J_χ-even and J_χ-odd parts. □

12.1. **Canonical choice of the reflection phase.** The phase χ can be fixed lobe by lobe without fitting the later fibres. Let $J_0 f = \overline{f(1-q)}$ and $u_V = Q_\perp W_{\beta_0}$. Choose

$$\boxed{2\chi_* = -\arg\langle u_V, J_0 u_V\rangle} \tag{12.9}$$

when the displayed inner product is nonzero. This choice minimizes

$$\|u_V - J_\chi u_V\| \tag{12.10}$$

over χ, hence minimizes the projected initial reflection defect.

Proof. Since $J_\chi = e^{2i\chi}J_0$,

$$\|u_V - J_\chi u_V\|^2 = 2\|u_V\|^2 - 2\,\mathrm{Re}\left(e^{2i\chi}\langle u_V, J_0 u_V\rangle\right). \tag{12.11}$$

The right-hand side is minimized by maximizing the real part. □

13. Exact modal phase structure of the reflection split

Let ϕ_m be the real shifted-Legendre basis, so

$$\phi_m(1-q) = (-1)^m \phi_m(q). \tag{13.1}$$

Define projected modal coefficients

$$x_m = \langle \phi_m, Q_\perp X \rangle, \qquad y_m = \langle \phi_m, Q_\perp Y \rangle. \tag{13.2}$$

Theorem 13.1 (Two real modal sequences)**.** *For every $m \geq 4$ there exist real numbers b_m and d_m such that*

$$\boxed{x_m = e^{i\chi} i^m b_m, \qquad y_m = e^{i\chi} i^{m+1} d_m.} \tag{13.3}$$

Consequently the full projected coefficient is

$$\boxed{a_m = x_m + y_m = e^{i\chi} i^m (b_m + i d_m), \qquad b_m, d_m \in \mathbb{R}.} \tag{13.4}$$

Proof. From $J_\chi X = X$, parity of ϕ_m, and (12.2),

$$\overline{x_m} = (-1)^m e^{-2i\chi} x_m. \tag{13.5}$$

This is equivalent to the first representation in (13.3). The relation $J_\chi Y = -Y$ similarly gives

$$\overline{y_m} = (-1)^{m+1} e^{-2i\chi} y_m, \tag{13.6}$$

which yields the second representation. □

This is stronger than merely saying that Y is the phase defect. All phase information is reduced exactly to the real ratios d_m/b_m. For two parameters α, β,

$$\overline{a_m(\alpha)} a_m(\beta) = b_m(\alpha) b_m(\beta) + d_m(\alpha) d_m(\beta) + i\big(b_m(\alpha) d_m(\beta) - d_m(\alpha) b_m(\beta)\big). \tag{13.7}$$

Thus a small Y means a small real sequence d_m relative to the symmetric-core sequence b_m.

There remains one sign issue even in the perfectly symmetric core: b_m is real, not automatically positive. A TP2 composition argument for the symmetric core therefore also requires a fixed modal sign choice $\sigma_m \in \{\pm 1\}$, independent of β, for which $\sigma_m b_m(\beta) > 0$ on the interval. This condition is explicit and numerically testable.

14. Plücker decomposition and perturbative aguteness

Set

$$\mathsf{X}_\beta = Q_\perp X_\beta, \qquad \mathsf{Y}_\beta = Q_\perp Y_\beta, \qquad u_\beta = \mathsf{X}_\beta + \mathsf{Y}_\beta. \tag{14.1}$$

Define the local bivector

$$\Delta_\beta = u_\beta \wedge u'_\beta. \tag{14.2}$$

Then

$$\boxed{\Delta_\beta = \Delta_\beta^{(+)} + E_\beta, \qquad \Delta_\beta^{(+)} = \mathsf{X}_\beta \wedge \mathsf{X}'_\beta,} \tag{14.3}$$

with

$$E_\beta = \mathsf{X}_\beta \wedge \mathsf{Y}'_\beta + \mathsf{Y}_\beta \wedge \mathsf{X}'_\beta + \mathsf{Y}_\beta \wedge \mathsf{Y}'_\beta. \tag{14.4}$$

Hence

$$\boxed{\|E_\beta\| \leq \|\mathsf{X}_\beta\| \left\|\mathsf{Y}'_\beta\right\| + \|\mathsf{Y}_\beta\| \left\|\mathsf{X}'_\beta\right\| + \|\mathsf{Y}_\beta\| \left\|\mathsf{Y}'_\beta\right\|.} \tag{14.5}$$

For two parameters define

$$A(\alpha,\beta) = \langle \Delta_\alpha, \Delta_\beta \rangle, \qquad A_+(\alpha,\beta) = \left\langle \Delta_\alpha^{(+)}, \Delta_\beta^{(+)} \right\rangle, \tag{14.6}$$

while

$$K_+(\alpha,\beta) = \langle \mathsf{X}_\alpha, \mathsf{X}_\beta \rangle. \tag{14.7}$$

The exterior-product determinant identity gives

$$\boxed{\mathcal{C}(\alpha,\beta) = \operatorname{Re}\frac{A(\alpha,\beta)}{K(\alpha,\beta)^2}.} \tag{14.8}$$

The reflection split gives a useful exact perturbation bound without asserting a sign theorem. Introduce

$$\eta = \frac{|K-K_+|}{|K_+|}, \qquad \varepsilon = \frac{|A-A_+|}{|K_+|^2}, \qquad R_+ = \frac{|A_+|}{|K_+|^2}, \tag{14.9}$$

whenever $K_+ \neq 0$. On the subregion $\eta < 1$, direct algebra gives

$$\boxed{\left|\frac{A}{K^2} - \frac{A_+}{K_+^2}\right| \leq \frac{\varepsilon + R_+\eta(2+\eta)}{(1-\eta)^2}.} \tag{14.10}$$

Indeed, write $K = K_+(1+z)$ with $|z| \leq \eta$, split the difference into the numerator defect and the denominator defect, and use

$$\left|\frac{1}{(1+z)^2} - 1\right| \leq \frac{\eta(2+\eta)}{(1-\eta)^2}. \tag{14.11}$$

Equation (14.10) is kept only as a quantitative comparison between the symmetric core and the full Rainbow bivector. A positive lower margin for the symmetric core and the required defect bound have not been proved for the full parameter region, so no curvature sign is inferred from this estimate in Version 5.

14.1. **Modal Pluecker identity and accumulated torsion.** Let

$$u_\beta = \sum_{m\geq 4} a_m(\beta)\phi_m \tag{14.12}$$

in a fixed real orthonormal basis of $\mathcal{P}_3^\perp$. For $j < k$, define the modal Pluecker coordinates

$$\boxed{\Delta_{jk}(\beta) = a_j(\beta)a_k'(\beta) - a_j'(\beta)a_k(\beta).} \tag{14.13}$$

The exterior Lagrange identity gives

$$\boxed{KK_{\alpha\beta} - K_\alpha K_\beta = \sum_{j<k} \overline{\Delta_{jk}(\alpha)}\,\Delta_{jk}(\beta).} \tag{14.14}$$

Consequently,

$$\boxed{\mathcal{C}(\alpha,\beta) = \operatorname{Re}\left[\frac{\displaystyle\sum_{j<k}\overline{\Delta_{jk}(\alpha)}\Delta_{jk}(\beta)}{K(\alpha,\beta)^2}\right].} \tag{14.15}$$

This replaces any need for mode-by-mode phase locking by a weaker projective aguteness condition on the complete Pluecker vector.

Where all displayed factors are nonzero, put

$$\Psi_{jk}(\alpha,\beta) = \arg\Delta_{jk}(\beta) - \arg\Delta_{jk}(\alpha) - 2\arg K(\alpha,\beta). \tag{14.16}$$

Then (14.15) becomes the exact cosine decomposition

$$\boxed{\mathcal{C}(\alpha,\beta) = \frac{1}{|K(\alpha,\beta)|^2}\sum_{j<k}|\Delta_{jk}(\alpha)|\,|\Delta_{jk}(\beta)|\cos\Psi_{jk}(\alpha,\beta).} \tag{14.17}$$

The termwise aguteness target, which is stronger than the exact weighted-sum requirement, is

$$|\Psi_{jk}(\alpha,\beta)| \leq \frac{\pi}{2} \qquad \text{for every contributing pair } j<k. \tag{14.18}$$

The exact target is weaker: only the weighted sum in (14.17) must be nonnegative.

The phase defect itself obeys a one-dimensional integral law. Since

$$\Delta'_{jk} = a_j a''_k - a''_j a_k, \tag{14.19}$$

we have, wherever $\Delta_{jk} \neq 0$,

$$\frac{\mathrm{d}}{\mathrm{d}\beta} \arg \Delta_{jk}(\beta) = \operatorname{Im} \frac{a_j a''_k - a''_j a_k}{a_j a'_k - a'_j a_k}. \tag{14.20}$$

For fixed α,

$$\frac{\mathrm{d}}{\mathrm{d}\beta} \arg K(\alpha,\beta) = \operatorname{Im} \frac{K_\beta(\alpha,\beta)}{K(\alpha,\beta)}. \tag{14.21}$$

Because $K(\alpha,\alpha) = \|u_\alpha\|^2 > 0$, one has $\Psi_{jk}(\alpha,\alpha) = 0$. Hence

$$\boxed{\Psi_{jk}(\alpha,\beta) = \int_\alpha^\beta \tau_{jk}^{(\alpha)}(s)\,\mathrm{d}s,} \tag{14.22}$$

where the projective modal torsion is

$$\boxed{\tau_{jk}^{(\alpha)}(s) = \operatorname{Im}\left[\frac{a_j a''_k - a''_j a_k}{a_j a'_k - a'_j a_k} - 2\frac{K_\beta(\alpha,s)}{K(\alpha,s)}\right].} \tag{14.23}$$

All quantities in (14.23) are internal Rainbow quantities. In particular,

$$a_m^{(r)}(\beta) = \langle \phi_m, Q_\perp L_n^r W_{\beta,n} \rangle\,, \qquad r = 0,1,2, \tag{14.24}$$

for the fixed transverse generator L_n.

The accumulated-torsion expression identifies a concrete open sign target. Termwise aguteness would follow from

$$\left| \int_\alpha^\beta \tau_{jk}^{(\alpha)}(s)\,\mathrm{d}s \right| \leq \frac{\pi}{2}, \tag{14.25}$$

but Version 5 does not assert (14.25). The exact requirement is weaker: the complete weighted cosine sum in (14.17) must be nonnegative after all modal cancellations are included.

On the diagonal, (14.14) reduces to

$$\boxed{\sum_{j<k} |\Delta_{jk}(\beta)|^2 = \|u_\beta\|^2 \left\|u'_\beta\right\|^2 - |\left\langle u_\beta, u'_\beta \right\rangle|^2,} \tag{14.26}$$

so the Pluecker formulation recovers $\mathcal{C}(\beta,\beta) = P(\beta)$ exactly. The unresolved extension problem is therefore the rotation of the Pluecker vector away from the diagonal, not a missing diagonal positivity statement.

14.2. **Projective stationarity and the genuine nonvanishing problem.** The standard nondegeneracy of the classical Pluecker pairing, the Klein quadric, or the Grassmannian does not imply that the specific Rainbow bivector $\Delta_\beta = u_\beta \wedge u'_\beta$ is nonzero. The vector Δ_β is decomposable by construction, and the zero bivector is precisely the projective-stationary case that disappears upon projectivizing the Grassmannian. No external Pluecker nondegeneracy theorem is used to exclude it.

On the regular set $u_\beta \neq 0$, projective stationarity has the exact equivalences

$$\boxed{\Delta_\beta = 0 \iff u'_\beta \in \mathbb{C}u_\beta \iff \Delta_{jk}(\beta) = 0 \text{ for every } j<k.} \tag{14.27}$$

Whenever two modal coefficients are nonzero, this gives the exact implication

$$\Delta_\beta = 0 \quad \Longrightarrow \quad \frac{\mathrm{d}}{\mathrm{d}\beta} \log \frac{|a_k(\beta)|^2}{|a_j(\beta)|^2} = 0. \tag{14.28}$$

Thus any strict adjacent modal-ratio inequality at a given parameter excludes projective stationarity there, but no such strict inequality is asserted globally in this paper.

There is also a quotient-internal necessary condition for a stationary point. Since $u = Q_\perp W$ and $u' = Q_\perp LW$, $\Delta_\beta = 0$ with $u_\beta \neq 0$ yields a scalar λ for which

$$LW_\beta - \lambda W_\beta \in \mathcal{P}_3. \tag{14.29}$$

Writing $D_q = \partial_q$ and differentiating four times gives

$$D_q^4(LW_\beta) = \lambda D_q^4 W_\beta. \tag{14.30}$$

Hence every projective-stationary point must satisfy

$$\boxed{\mathscr{N}_\beta(q) := D_q^4 W_\beta \, D_q^5(LW_\beta) - D_q^5 W_\beta \, D_q^4(LW_\beta) \equiv 0.} \tag{14.31}$$

Equation (14.31) is a necessary condition, not a claimed exclusion theorem. It supplies an internal Rainbow obstruction that can be attacked without invoking a different Pluecker nondegeneracy result.

The defect parameters have direct norm bounds. From $u = \mathsf{X} + \mathsf{Y}$,

$$|K - K_+| \le \|\mathsf{X}_\alpha\| \, \|\mathsf{Y}_\beta\| + \|\mathsf{Y}_\alpha\| \, \|\mathsf{X}_\beta\| + \|\mathsf{Y}_\alpha\| \, \|\mathsf{Y}_\beta\| \, . \tag{14.32}$$

Similarly, (14.5) and Cauchy–Schwarz give

$$|A - A_+| \le \|E_\alpha\| \left\|\Delta_\beta^{(+)}\right\| + \left\|\Delta_\alpha^{(+)}\right\| \|E_\beta\| + \|E_\alpha\| \, \|E_\beta\| \, . \tag{14.33}$$

Thus the reflection split reduces the comparison to a symmetric-core contribution and explicit Y, Y' defects, without assigning their net sign.

15. Internal bound on the antisymmetric forcing

The reflection asymmetry of the generator admits an exact internal estimate. By the mean-value theorem and Lemma 2.2,

$$\begin{aligned} |a_-(q)| &= \frac{1}{2}|a(q) - a(1-q)| \\ &\le \frac{1}{2} \sup_{0 \le s \le 1} |a'(s)| \\ &= \frac{1}{2} \sup_{\gamma_n \le t \le \gamma_{n+1}} \left| \frac{\vartheta''(t)}{\vartheta'(t)} \right| . \end{aligned} \tag{15.1}$$

Hence, on H^1,

$$\boxed{\|L_- f\|_{L^2} \le \frac{1}{2} \sup_{[\gamma_n, \gamma_{n+1}]} \left| \frac{\vartheta''}{\vartheta'} \right| \|\partial_q f\|_{L^2} \, .} \tag{15.2}$$

Using the standard large-t derivatives

$$\vartheta'(t) = \frac{1}{2} \log \frac{t}{2\pi} + O(t^{-2}), \qquad \vartheta''(t) = \frac{1}{2t} + O(t^{-3}), \tag{15.3}$$

the coefficient in (15.2) is

$$O\!\left(\frac{1}{t \log t} \right) \tag{15.4}$$

on high lobes. This is a different quantity from the formal commutator size in (9.9).

The second equation in (12.8) has the variation-of-constants form

$$\boxed{Y_\beta = S_+(\beta - \beta_0) Y_{\beta_0} + \int_{\beta_0}^{\beta} S_+(\beta - s) L_- X_s \, \mathrm{d}s,} \tag{15.5}$$

whenever the propagator S_+ of L_+ is defined on the chosen regularity class. Thus the two quantities entering the defect are exactly the minimized initial component Y_{β_0} and the controlled antisymmetric coefficient a_-. No third forcing term is present.

16. Pólya–Chebyshev module

The modal route is most naturally phrased for the nonnegative sequence

$$p_m(\beta) = |c_m(\beta)|^2, \qquad m \geq 4. \tag{16.1}$$

For a finite truncation $4 \leq m \leq N$, let

$$\boxed{G_N(\beta; z) = \sum_{r=0}^{N-4} p_{r+4}(\beta) z^r.} \tag{16.2}$$

A classical theorem of Aissen–Edrei–Schoenberg–Whitney characterizes finite Pólya-frequency sequences through generating polynomials with only real nonpositive zeros; see [7, 8]. We record the exact coefficient identity that links such a root motion to the corrected adjacent modal-ratio sign.

16.1. **Exact algebraic root-flow reduction.** For a factorization

$$G_N(\beta; z) = C(\beta) \prod_{\nu=1}^{R} (1 + \lambda_\nu(\beta) z), \qquad C > 0, \tag{16.3}$$

with real positive λ_ν, the coefficients satisfy

$$p_{r+4} = C e_r(\lambda_1, \ldots, \lambda_R). \tag{16.4}$$

For each variable λ_i,

$$\boxed{\frac{\partial}{\partial \lambda_i} \log \frac{e_{r+1}}{e_r} = \frac{(e_r^{(i)})^2 - e_{r-1}^{(i)} e_{r+1}^{(i)}}{e_r e_{r+1}} \geq 0,} \tag{16.5}$$

where the last inequality is Newton's inequality. Consequently, monotone motion $\lambda_i' \geq 0$ would force the corrected adjacent ratio sign

$$\frac{\mathrm{d}}{\mathrm{d}\beta} \log \frac{p_{m+1}}{p_m} \geq 0. \tag{16.6}$$

This is an exact algebraic reduction, not a statement that the Rainbow polynomials possess such a root flow. Establishing that motion for the actual G_N remains open.

If $z_\nu(\beta) = -1/\lambda_\nu(\beta)$, then $\lambda_\nu' \geq 0$ is equivalent to $z_\nu' \geq 0$: every negative root moves toward the origin. At a simple root,

$$\boxed{z_\nu'(\beta) = -\frac{\partial_\beta G_N(\beta; z_\nu)}{\partial_z G_N(\beta; z_\nu)}.} \tag{16.7}$$

Thus the Pólya route is reduced to a concrete sign problem.

16.2. **Hermitian form for the root speed.** Let $c(\beta)$ denote the full modal vector and define the diagonal finite-rank operator

$$E_N(z) = \mathrm{diag}(0, 0, 0, 0, 1, z, z^2, \ldots, z^{N-4}, 0, \ldots). \tag{16.8}$$

Then

$$G_N(\beta; z) = c(\beta)^* E_N(z) c(\beta). \tag{16.9}$$

Using $c' = -iDc$,

$$\boxed{\partial_\beta G_N = c^* \mathscr{H}_N(z) c, \qquad \mathscr{H}_N(z) = i\big(D^T E_N(z) - E_N(z) D\big),} \tag{16.10}$$

where $\mathscr{H}_N(z)$ is Hermitian for real z. Hence

$$\boxed{z_\nu' = -\frac{c^* \mathscr{H}_N(z_\nu) c}{c^* E_N'(z_\nu) c}.} \tag{16.11}$$

The Pólya root-flow question is therefore reduced to signs of explicit Hermitian forms along the actual transverse orbit.

For $z = -x < 0$,

$$G_N(-x) = (p_4 + p_6x^2 + p_8x^4 + \cdots) - (p_5x + p_7x^3 + p_9x^5 + \cdots). \tag{16.12}$$

Thus every negative root is an equality point between the even- and odd-mode sums. This matches the dominant opposite-parity coupling produced by the derivative operator in the Legendre basis, but does not by itself determine the direction of root motion.

17. The phase bridge: modulus kernel and coherence factor

Write the high-mode coefficient as

$$a_m(\beta) = b_m(\beta)e^{i\theta_m(\beta)}, \qquad b_m = |a_m|. \tag{17.1}$$

Define

$$H(\alpha, \beta) = \sum_{m\geq 4} b_m(\alpha)b_m(\beta) \tag{17.2}$$

and

$$\boxed{\rho(\alpha, \beta) = \frac{|K(\alpha, \beta)|}{H(\alpha, \beta)} \leq 1.} \tag{17.3}$$

Then

$$|K| = H\rho. \tag{17.4}$$

Karlin's classical composition formula identifies the transfer mechanism from modal TP2 to TP2 of H under the usual summability conditions; Version 5 does not assert that the Rainbow modal kernel is TP2. The separate phase/coherence factor is ρ.

There is an exact favorable fact on the diagonal. Put

$$w_m = \frac{b_m^2}{\sum_j b_j^2}, \qquad s_m = \partial_\beta \log b_m^2. \tag{17.5}$$

Then the projective speed decomposes as

$$\boxed{P = \frac{1}{4}\operatorname{Var}_w(s_m) + \operatorname{Var}_w(\theta'_m).} \tag{17.6}$$

The first term is the diagonal mixed curvature of H. Consequently

$$\boxed{\partial_\alpha\partial_\beta \log \rho(\alpha, \beta)|_{\alpha=\beta} = \operatorname{Var}_w(\theta'_m) \geq 0.} \tag{17.7}$$

Thus phase variation contributes nonnegatively to the diagonal curvature. The unresolved issue is whether a corresponding lower bound persists off the diagonal.

The reflection decomposition sharpens this bridge. Equation (13.4) expresses every phase through two real sequences b_m, d_m. The perfect symmetric core is $d_m = 0$; the defect is exactly the size and variation of d_m. Equation (14.10) quantifies how the reflection defect changes the full Pluecker curvature relative to the symmetric core; no sign conclusion is attached to that comparison without further Rainbow estimates.

18. Current analytic status and surviving routes

The exact sign problem is

$$\boxed{M_n'(\beta) = 2(1 - M_n(\beta)) \int_{1/2}^{\beta} \mathcal{C}_n(\alpha, \beta)\, \mathrm{d}\alpha.} \tag{18.1}$$

Version 5 does not claim an analytic positive sign for this integral on any nontrivial β interval. The diagonal identity (6.8) and the finite positive scans are not promoted into an off-diagonal theorem.

Two routes are closed by unconditional calculations. First, the raw modal generator in a real basis is purely imaginary and is not a real Metzler system. Second, the constant-coefficient

Legendre transport model admits no fixed diagonal phase gauge that makes all nonzero modal couplings cooperative. Those strategies are not used as proof routes.

The surviving analytic information is more specific. The coherence equation (11.2) is closed and retains all modal phases. The reflection system (12.8) reduces those phases to two real modal sequences and isolates the antisymmetric forcing. The exact Pluecker identity (14.15) reduces mixed curvature to the projective orientation of the modal bivector, while (14.22) converts that orientation into accumulated one-dimensional torsion. The Pólya calculation (16.5) identifies a separate magnitude-order mechanism but does not supply the missing phase sign by itself. Projective stationarity is isolated by (14.27) and the internal obstruction (14.31); its exclusion for every Rainbow parameter remains unresolved.

The numerical record therefore has a precise status. For the tested orthogonal-cubic data, M' was nonnegative at all 64,560 sampled points and the direct $\mathcal{C}$ scan on lobes 2, 8, and 25 reported no negative sampled values. These are finite observations. The global analytic target remains the sign of the integral in (18.1), together with the regularity and nonvanishing information required to use the logarithmic kernel formulas globally.

19. Bibliographic position

General Gram positivity does not imply TP2. Karlin's monograph develops total positivity and variation-diminishing kernels [3]; Aronszajn develops general reproducing-kernel Hilbert theory [2]; Burbea studies conditions under which reproducing kernels are totally positive rather than treating total positivity as automatic [4, 5]. Belton–Guillot–Khare–Putinar further study preservers of totally positive kernels and Pólya frequency functions [6].

The generating-polynomial route belongs to the classical Pólya-frequency program. Aissen–Edrei–Schoenberg–Whitney relate totally positive sequences to their generating functions [7]; Schoenberg develops Pólya frequency functions and total positivity [8]. Modern stability-preserver theory, including Borcea–Brändén [9], provides a broader setting for operators preserving real-rootedness and stability. None of these general results makes the present projected kernel TP2 merely from its Gram representation; the lobe-specific reflection and transverse dynamics remain essential.

20. Reproducibility package

The `RAINBOW_V5` archive is intentionally restricted to material used in this direction. It contains the principal TeX and compiled PDF, the five retained PNG figures, three MATLAB scripts, the fixed ordinate file `zeross.bin`, the two preceding console logs, the supplied pointwise-curvature report, a README, and a SHA-256 manifest. No legacy Python is included. The unused `zLamb.bin` file is also omitted because none of the retained scripts reads it.

The third MATLAB script, `RAINBOW_REFLECTION_POLYA_AUDIT.m`, reconstructs the direct $\mathcal{C}$ scan and adds the canonical reflection gauge, Y/Y' defect ratios, corrected adjacent Chebyshev determinants, and finite Pólya-root diagnostics. These diagnostics are exploratory tests of the unresolved Rainbow sign mechanisms; no unexecuted diagnostic is promoted to a mathematical result.

References

[1] E. C. Titchmarsh, revised by D. R. Heath-Brown, *The Theory of the Riemann Zeta-Function*, 2nd ed., Oxford University Press, 1986.

[2] N. Aronszajn, Theory of reproducing kernels, *Transactions of the American Mathematical Society* **68** (1950), 337–404. DOI: 10.1090/S0002-9947-1950-0051437-7.

[3] S. Karlin, *Total Positivity, Vol. I*, Stanford University Press, Stanford, 1968.

[4] J. Burbea, Total positivity and reproducing kernels, *Pacific Journal of Mathematics* **55** (1974), 343–359. DOI: 10.2140/pjm.1974.55.343.

[5] J. Burbea, Total positivity of certain reproducing kernels, *Pacific Journal of Mathematics* **67** (1976), 101–130. DOI: 10.2140/pjm.1976.67.101.

[6] A. Belton, D. Guillot, A. Khare, and M. Putinar, Preservers of totally positive kernels and Pólya frequency functions, *Mathematics Research Reports* **3** (2022), 35–56. DOI: 10.5802/mrr.12.

[7] M. Aissen, A. Edrei, I. J. Schoenberg, and A. Whitney, On the generating functions of totally positive sequences, *Proceedings of the National Academy of Sciences USA* **37** (1951), 303–307. DOI: 10.1073/pnas.37.5.303.
[8] I. J. Schoenberg, On Pólya frequency functions. I. The totally positive functions and their Laplace transforms, *Journal d'Analyse Mathématique* **1** (1951), 331–374. DOI: 10.1007/BF02790092.
[9] J. Borcea and P. Brändén, Pólya–Schur master theorems for circular domains and their boundaries, *Annals of Mathematics* **170** (2009), 465–492. DOI: 10.4007/annals.2009.170.465.

Appendix A. MATLAB: five-quotient rigidity audit

```
%% VIOLET_TRANSVERSE_QUOTIENT_RIGIDITY_AUDIT.m
% Pure MATLAB.  Transverse/Violet quotient-rigidity audit.
%
% Violet is beta = 1/2.  The supplied ordinates are used only to define
% the fixed Violet zero-to-zero lobe boundaries.  No zeros are searched or
% transported as beta varies.
%
% For each lobe n, a fixed phase coordinate q -> t_n(q) is built on the
% Violet fibre.  Every Transverse fibre W_{beta,n}(q) is evaluated on the same
% t_n(q).  Five beta-independent linear quotients Q are compared:
%   1 HERMITE_JET1       values + first derivatives at q=0,1
%   2 ENDPOINT_VALUES    values only
%   3 HERMITE_JET2       values + first + second derivatives
%   4 MINIMAL_BOUNDARY_SPLINE  four fixed anchors
%   5 ORTHOGONAL_CUBIC   weighted L2 projection off span{1,q,q^2,q^3}
%
% Projective defect:
%   M_n^Q(beta) = 1 - |<r^Q_{V,n},r^Q_{beta,n}>|^2,
%   r = QW/||QW||.
%
% The script reproduces the quotient-rigidity, same-lobe identity, and
% cross-quotient concordance diagnostics reported in the manuscript.

clear; clc; close all;
rng(20260825,'twister');

%% Paths and configuration
SCRIPT_DIR = fileparts(mfilename('fullpath'));
ROOT = fileparts(SCRIPT_DIR);

cfg = struct;
cfg.ZeroFile = fullfile(ROOT,'data','zeross.bin');
cfg.NLobes = 240;
cfg.Nq = 81;
cfg.NBeta = 49;
cfg.DeltaMin = 1e-4;
cfg.DeltaMax = 0.5;
cfg.EMOrder = 8;
cfg.InitialCutoff = 512;
cfg.MaxCutoff = 4096;
cfg.TransverseAuditTol = 2e-9;
cfg.TChunk = 900;
cfg.MonotonicRelTol = 2e-3;
cfg.MonotonicAbsTol = 2e-7;
cfg.BetaOrderNullReps = 3000;
cfg.LobeIdentityNullReps = 500;
cfg.NIdentityAnchors = 8;
cfg.SplineAnchorLeft = 0.10;
cfg.SplineAnchorRight = 0.90;
cfg.OutputDir = fullfile(ROOT,'outputs','quotient_rigidity');
if ~exist(cfg.OutputDir,'dir'), mkdir(cfg.OutputDir); end

quotientNames = { ...
    'HERMITE_JET1'; ...
    'ENDPOINT_VALUES'; ...
    'HERMITE_JET2'; ...
    'MINIMAL_BOUNDARY_SPLINE'; ...
    'ORTHOGONAL_CUBIC'};
NQ = numel(quotientNames);

```

```
fprintf('\nVIOLET TRANSVERSE QUOTIENT RIGIDITY AUDIT\n');
fprintf('------------------------------------------------------------\n');
fprintf('Violet lobes            : %d\n',cfg.NLobes);
fprintf('q points / lobe         : %d\n',cfg.Nq);
fprintf('Transverse beta fibres     : %d\n',cfg.NBeta);
fprintf('quotient families       : %d\n',NQ);
fprintf('beta-order nulls        : %d\n',cfg.BetaOrderNullReps);
fprintf('lobe-identity nulls     : %d\n',cfg.LobeIdentityNullReps);

%% Load fixed Violet boundaries
fid = fopen(cfg.ZeroFile,'rb');
if fid < 0, error('Cannot open %s.',cfg.ZeroFile); end
gamma = fread(fid,cfg.NLobes+1,'double');
fclose(fid);
if numel(gamma) < cfg.NLobes+1, error('Not enough Violet boundaries.'); end
gamma = gamma(:);

%% Fixed phase projection
q = linspace(0,1,cfg.Nq);
wq = trapezoid_weights(q);
tGrid = violet_phase_grid(gamma,q);
Tall = reshape(tGrid.',1,[]);

fprintf('\nFIXED VIOLET PHASE PROJECTION\n');
fprintf('------------------------------------------------------------\n');
fprintf('max left endpoint residual  = %.3e\n',max(abs(tGrid(:,1)-gamma(1:end-1))));
fprintf('max right endpoint residual = %.3e\n',max(abs(tGrid(:,end)-gamma(2:end))));

%% Transverse beta grid
positiveDelta = logspace(log10(cfg.DeltaMin),log10(cfg.DeltaMax),cfg.NBeta-1).';
delta = [0; positiveDelta];
beta = 0.5 + delta;
beta(end) = 1; delta(end) = 0.5;
NB = numel(beta); NL = cfg.NLobes;

%% Frozen spline anchors
[~,iL] = min(abs(q-cfg.SplineAnchorLeft));
[~,iR] = min(abs(q-cfg.SplineAnchorRight));
splineAnchorIndex = unique([1 iL iR cfg.Nq]);
if numel(splineAnchorIndex) ~= 4, error('Spline anchors collapsed.'); end

%% Euler--Maclaurin evaluator audit
fprintf('\nTRANSVERSE NUMERICAL AUDIT\n');
fprintf('------------------------------------------------------------\n');
betaAudit = [0.5;0.5001;0.51;0.60;0.80;1.00];
pickT = round(linspace(1,numel(Tall),19));
tAudit = Tall(pickT);
Ncut = cfg.InitialCutoff;
while true
    Za = transverse_matrix(betaAudit,tAudit,Ncut,cfg.EMOrder,cfg.TChunk);
    Zb = transverse_matrix(betaAudit,tAudit,2*Ncut,cfg.EMOrder,cfg.TChunk);
    err = max(abs(Za(:)-Zb(:))./max(1,abs(Zb(:))));
    fprintf('cutoff %5d vs %5d : relative error %.3e\n',Ncut,2*Ncut,err);
    if err <= cfg.TransverseAuditTol, break; end
    Ncut = 2*Ncut;
    if 2*Ncut > cfg.MaxCutoff, error('Transverse evaluator failed audit.'); end
end
fprintf('accepted cutoff = %d\n',Ncut);

%% Complete Transverse
fprintf('\nCONSTRUCTING COMPLETE TRANSVERSE PROJECTION\n');
tic;
Transverse = transverse_matrix(beta,Tall,Ncut,cfg.EMOrder,cfg.TChunk);
fprintf('elapsed = %.2f s\n',toc);

WVraw = reshape(Transverse(1,:),cfg.Nq,NL).';
fprintf('\nVIOLET BOUNDARY AUDIT\n');
fprintf('------------------------------------------------------------\n');
fprintf('max boundary residual    = %.3e\n',max(abs([WVraw(:,1);WVraw(:,end)])));
fprintf('median boundary residual = %.3e\n',median(abs([WVraw(:,1);WVraw(:,end)])));
```

```
WVraw(:,1)=0; WVraw(:,end)=0;

identityAnchorIndex = unique(round(linspace(2,NB,cfg.NIdentityAnchors)));
Nanchor = numel(identityAnchorIndex);

%% Storage
DefectByFamily = cell(NQ,1);
MedianCurveByFamily = nan(NB,NQ);
LobeStepFractionByFamily = nan(NL,NQ);
FullMonotoneMatrix = false(NL,NQ);
IdentityObserved = nan(Nanchor,NQ);
IdentityNullMedian = nan(Nanchor,NQ);
IdentityP = nan(Nanchor,NQ);
IdentityRatio = nan(Nanchor,NQ);
OrderNullRho = nan(cfg.BetaOrderNullReps,NQ);

Summary = table('Size',[NQ 11], ...
    'VariableTypes',{'string','double','double','double','double','double','double','double','double','
     double','double'}, ...
    'VariableNames',{'quotient','violetDefect','beta1Defect','spearmanBeta','orderNullP', ...
    'aggregateStepFraction','medianLobeStepFraction','fullMonotoneLobes', ...
    'fullMonotoneLobeFraction','maximumDownwardStep','beta1IdentityRatio'});

%% Main loop
for jq = 1:NQ
    qName = quotientNames{jq};
    fprintf('\nQUOTIENT %d / %d : %s\n',jq,NQ,qName);
    fprintf('------------------------------------------------------------\n');

    rhoV = apply_quotient(WVraw,q,wq,qName,splineAnchorIndex);
    [rV,normV,goodV] = normalize_rows(rhoV,wq);
    fprintf('usable Violet lobes = %d / %d\n',nnz(goodV),NL);
    fprintf('median residual norm = %.8e\n',median(normV(goodV)));

    Rnormalized = complex(nan(NB,NL,cfg.Nq));
    M = nan(NB,NL);

    for ib = 1:NB
        W = reshape(Transverse(ib,:),cfg.Nq,NL).';
        if ib == 1, W(:,1)=0; W(:,end)=0; end
        rho = apply_quotient(W,q,wq,qName,splineAnchorIndex);
        [r,~,good] = normalize_rows(rho,wq);
        good = good & goodV;
        Rnormalized(ib,:,:) = r;
        inner = sum(conj(rV).*r.*wq.',2);
        align2 = min(max(real(abs(inner).^2),0),1);
        defect = 1-align2;
        defect(~good)=NaN;
        M(ib,:) = defect.';
        if ib==1, M(ib,good)=0; end
    end

    DefectByFamily{jq}=M;
    medianCurve = row_median_finite(M);
    MedianCurveByFamily(:,jq)=medianCurve;
    report = monotonicity_report(medianCurve,M,cfg);
    LobeStepFractionByFamily(:,jq)=report.lobeStepFraction;
    FullMonotoneMatrix(:,jq)=report.fullLobeMask;

    observedRho = spearman_manual(beta,medianCurve);
    for rr=1:cfg.BetaOrderNullReps
        OrderNullRho(rr,jq)=spearman_manual(beta,medianCurve(randperm(NB)));
    end
    orderP=(1+sum(OrderNullRho(:,jq)>=observedRho))/(cfg.BetaOrderNullReps+1);

    fprintf('\nSAME-LOBE IDENTITY NULL\n');
    fprintf('------------------------------------------------------------\n');
    for ia=1:Nanchor
        ib=identityAnchorIndex(ia);
        rBeta=squeeze(Rnormalized(ib,:,:));
```

```
        observed=median_finite(M(ib,:));
        nullValues=nan(cfg.LobeIdentityNullReps,1);
        for rr=1:cfg.LobeIdentityNullReps
            perm=randperm(NL);
            innerNull=sum(conj(rV(perm,:)).*rBeta.*wq.',2);
            wrongDefect=1-min(max(abs(innerNull).^2,0),1);
            nullValues(rr)=median_finite(wrongDefect);
        end
        nullMedian=median(nullValues);
        pIdentity=(1+sum(nullValues<=observed))/(cfg.LobeIdentityNullReps+1);
        IdentityObserved(ia,jq)=observed;
        IdentityNullMedian(ia,jq)=nullMedian;
        IdentityP(ia,jq)=pIdentity;
        IdentityRatio(ia,jq)=nullMedian/max(observed,realmin);
        fprintf('beta %.7f | same %.6e | wrong %.6e | ratio %.3f | p %.6g\n', ...
            beta(ib),observed,nullMedian,IdentityRatio(ia,jq),pIdentity);
    end

    fprintf('\nMONOTONICITY\n');
    fprintf('------------------------------------------------------------\n');
    fprintf('Violet defect                = %.8e\n',medianCurve(1));
    fprintf('beta=1 defect                = %.8e\n',medianCurve(end));
    fprintf('Spearman(beta,M)             = %.10f\n',observedRho);
    fprintf('beta-order null p            = %.8g\n',orderP);
    fprintf('aggregate monotone steps     = %.6f\n',report.aggregateStepFraction);
    fprintf('median lobe monotone steps   = %.6f\n',report.medianLobeStepFraction);
    fprintf('fully monotone lobes         = %d / %d\n',nnz(report.fullLobeMask),NL);
    fprintf('fully monotone lobe fraction = %.6f\n',report.fullLobeFraction);
    fprintf('largest downward step        = %.8e\n',report.maximumDownwardStep);

    Summary.quotient(jq)=string(qName);
    Summary.violetDefect(jq)=medianCurve(1);
    Summary.beta1Defect(jq)=medianCurve(end);
    Summary.spearmanBeta(jq)=observedRho;
    Summary.orderNullP(jq)=orderP;
    Summary.aggregateStepFraction(jq)=report.aggregateStepFraction;
    Summary.medianLobeStepFraction(jq)=report.medianLobeStepFraction;
    Summary.fullMonotoneLobes(jq)=nnz(report.fullLobeMask);
    Summary.fullMonotoneLobeFraction(jq)=report.fullLobeFraction;
    Summary.maximumDownwardStep(jq)=report.maximumDownwardStep;
    Summary.beta1IdentityRatio(jq)=IdentityRatio(end,jq);
end

%% Cross-quotient rigidity
allFamilyMonotone=all(FullMonotoneMatrix,2);
NallFamily=nnz(allFamilyMonotone);
NpassedByLobe=sum(FullMonotoneMatrix,2);
MedianCurveSpearman=nan(NQ,NQ);
FullGeometrySpearman=nan(NQ,NQ);
for i=1:NQ
    for j=1:NQ
        MedianCurveSpearman(i,j)=spearman_manual(MedianCurveByFamily(:,i),MedianCurveByFamily(:,j));
        FullGeometrySpearman(i,j)=spearman_manual(DefectByFamily{i}(:),DefectByFamily{j}(:));
    end
end

fprintf('\nCROSS-QUOTIENT RIGIDITY\n');
fprintf('------------------------------------------------------------\n');
fprintf('lobes fully monotone under EVERY quotient: %d / %d = %.8f\n', ...
    NallFamily,NL,NallFamily/NL);
fprintf('\nPAIRWISE SPEARMAN OF MEDIAN beta CURVES\n'); disp(MedianCurveSpearman);
fprintf('\nPAIRWISE SPEARMAN OF FULL beta x LOBE GEOMETRY\n'); disp(FullGeometrySpearman);

%% Export tables
Tlobe=table((1:NL).',NpassedByLobe,allFamilyMonotone, ...
    'VariableNames',{'violetLobe','numberOfMonotoneQuotients','monotoneUnderEveryQuotient'});
for jq=1:NQ
    vn=matlab.lang.makeValidName(['stepFraction_' quotientNames{jq}]);
    Tlobe.(vn)=LobeStepFractionByFamily(:,jq);
end
```

```
writetable(Tlobe,fullfile(cfg.OutputDir,'LOBE_BY_LOBE_QUOTIENT_RIGIDITY.csv'));
writetable(Summary,fullfile(cfg.OutputDir,'QUOTIENT_RIGIDITY_SUMMARY.csv'));

Tidentity=table('Size',[Nanchor*NQ 6], ...
    'VariableTypes',{'double','string','double','double','double','double'}, ...
    'VariableNames',{'beta','quotient','sameLobeDefect','wrongLobeMedian','wrongOverSameRatio','lowerTailP'});
row=0;
for ia=1:Nanchor
    for jq=1:NQ
        row=row+1;
        Tidentity.beta(row)=beta(identityAnchorIndex(ia));
        Tidentity.quotient(row)=string(quotientNames{jq});
        Tidentity.sameLobeDefect(row)=IdentityObserved(ia,jq);
        Tidentity.wrongLobeMedian(row)=IdentityNullMedian(ia,jq);
        Tidentity.wrongOverSameRatio(row)=IdentityRatio(ia,jq);
        Tidentity.lowerTailP(row)=IdentityP(ia,jq);
    end
end
writetable(Tidentity,fullfile(cfg.OutputDir,'SAME_LOBE_IDENTITY_ACROSS_QUOTIENTS.csv'));

%% Figures
f=figure('Name','Transverse quotient rigidity','NumberTitle','off');
for jq=1:NQ
    semilogy(beta(2:end),max(MedianCurveByFamily(2:end,jq),realmin),'LineWidth',1.4,'DisplayName',
     quotientNames{jq}); hold on;
end
xlabel('\beta'); ylabel('median projective defect M');
title('Transverse/Violet projective defect under five quotients');
legend('Location','best','Interpreter','none'); grid on;
save_figure_png(f,fullfile(cfg.OutputDir,'fig01_projective_defect_all_quotients.png'));

f=figure('Name','Normalized quotient rigidity','NumberTitle','off');
for jq=1:NQ
    y=MedianCurveByFamily(:,jq)/max(MedianCurveByFamily(end,jq),realmin);
    plot(beta,y,'LineWidth',1.5,'DisplayName',quotientNames{jq}); hold on;
end
xlabel('\beta'); ylabel('M(\beta) / M(1)');
title('Normalized Transverse deformation across quotient choices');
legend('Location','best','Interpreter','none'); grid on;
save_figure_png(f,fullfile(cfg.OutputDir,'fig02_normalized_defect_curves.png'));

lobeShow=min(40,NL);
f=figure('Name','One lobe across quotients','NumberTitle','off');
for jq=1:NQ
    y=DefectByFamily{jq}(:,lobeShow);
    semilogy(beta(2:end),max(y(2:end),realmin),'LineWidth',1.4,'DisplayName',quotientNames{jq}); hold on;
end
xlabel('\beta'); ylabel(sprintf('M_{%d}(\beta)',lobeShow));
title(sprintf('Same Violet lobe %d under five boundary quotients',lobeShow));
legend('Location','best','Interpreter','none'); grid on;
save_figure_png(f,fullfile(cfg.OutputDir,'fig03_lobe40_all_quotients.png'));

save(fullfile(cfg.OutputDir,'VIOLET_TRANSVERSE_QUOTIENT_RIGIDITY_COMPLETE.mat'), ...
    'cfg','gamma','q','wq','tGrid','beta','delta','quotientNames', ...
    'DefectByFamily','MedianCurveByFamily','LobeStepFractionByFamily', ...
    'FullMonotoneMatrix','IdentityObserved','IdentityNullMedian','IdentityP', ...
    'IdentityRatio','MedianCurveSpearman','FullGeometrySpearman','Summary','-v7.3');

fprintf('\nAUDIT COMPLETE\n');

%% Local functions
function rho=apply_quotient(W,q,wq,qName,splineAnchorIndex)
    switch upper(qName)
        case 'HERMITE_JET1', rho=quotient_hermite_jet1(W,q);
        case 'ENDPOINT_VALUES', rho=quotient_endpoint_values(W,q);
        case 'HERMITE_JET2', rho=quotient_hermite_jet2(W,q);
        case 'MINIMAL_BOUNDARY_SPLINE', rho=quotient_boundary_spline(W,q,splineAnchorIndex);
        case 'ORTHOGONAL_CUBIC', rho=quotient_orthogonal_cubic(W,q,wq);
        otherwise, error('Unknown quotient %s.',qName);
```

```
    end
end

function rho=quotient_hermite_jet1(W,q)
    [d0,d1]=endpoint_first_derivatives(W,q); y0=W(:,1); y1=W(:,end); x=q(:).';
    h00=2*x.^3-3*x.^2+1; h10=x.^3-2*x.^2+x; h01=-2*x.^3+3*x.^2; h11=x.^3-x.^2;
    H=y0.*h00+d0.*h10+y1.*h01+d1.*h11; rho=W-H; rho(:,1)=0; rho(:,end)=0;
end

function rho=quotient_endpoint_values(W,q)
    x=q(:).'; H=W(:,1).*(1-x)+W(:,end).*x; rho=W-H; rho(:,1)=0; rho(:,end)=0;
end

function rho=quotient_hermite_jet2(W,q)
    [d0,d1]=endpoint_first_derivatives(W,q); [dd0,dd1]=endpoint_second_derivatives(W,q);
    y0=W(:,1); y1=W(:,end);
    A=[1 0 0 0 0 0; 0 1 0 0 0 0; 0 0 2 0 0 0; 1 1 1 1 1 1; 0 1 2 3 4 5; 0 0 2 6 12 20];
    C=A\[y0.';d0.';dd0.';y1.';d1.';dd1.']; x=q(:);
    V=[ones(size(x)) x x.^2 x.^3 x.^4 x.^5]; H=(V*C).'; rho=W-H; rho(:,1)=0; rho(:,end)=0;
end

function rho=quotient_boundary_spline(W,q,idx)
    H=complex(zeros(size(W))); qa=q(idx);
    for n=1:size(W,1), H(n,:)=spline(qa,W(n,idx),q); end
    rho=W-H;
end

function rho=quotient_orthogonal_cubic(W,q,wq)
    x=q(:); B=[ones(size(x)) x x.^2 x.^3]; G=B.'*(B.*wq); RHS=B.'*(W.'.*wq); C=G\RHS; H=(B*C).'; rho=W-H;
end

function [d0,d1]=endpoint_first_derivatives(W,q)
    h=q(2)-q(1);
    d0=(-25*W(:,1)+48*W(:,2)-36*W(:,3)+16*W(:,4)-3*W(:,5))/(12*h);
    d1=(25*W(:,end)-48*W(:,end-1)+36*W(:,end-2)-16*W(:,end-3)+3*W(:,end-4))/(12*h);
end

function [dd0,dd1]=endpoint_second_derivatives(W,q)
    h=q(2)-q(1);
    dd0=(35*W(:,1)-104*W(:,2)+114*W(:,3)-56*W(:,4)+11*W(:,5))/(12*h^2);
    dd1=(35*W(:,end)-104*W(:,end-1)+114*W(:,end-2)-56*W(:,end-3)+11*W(:,end-4))/(12*h^2);
end

function [R,nrm,good]=normalize_rows(X,wq)
    nrm=sqrt(sum(abs(X).^2.*wq.',2)); good=isfinite(nrm)&nrm>1e-12; R=complex(nan(size(X))); R(good,:)=X(
     good,:)./nrm(good);
end

function report=monotonicity_report(curve,M,cfg)
    dc=diff(curve); sc=max(abs(curve(1:end-1)),abs(curve(2:end))); tol=cfg.MonotonicAbsTol+cfg.
     MonotonicRelTol.*sc;
    good=isfinite(dc)&isfinite(tol); pass=dc>=-tol;
    report.aggregateStepFraction=mean(pass(good));
    if any(good), report.maximumDownwardStep=max(max(0,-dc(good))); else, report.maximumDownwardStep=NaN;
     end
    NL=size(M,2); frac=nan(NL,1); full=false(NL,1);
    for n=1:NL
        y=M(:,n); dy=diff(y); sc=max(abs(y(1:end-1)),abs(y(2:end))); tol=cfg.MonotonicAbsTol+cfg.
     MonotonicRelTol.*sc;
        good=isfinite(dy)&isfinite(tol); if ~any(good), continue; end
        pp=dy(good)>=-tol(good); frac(n)=mean(pp); full(n)=all(pp);
    end
    report.lobeStepFraction=frac; report.fullLobeMask=full; report.medianLobeStepFraction=median_finite(
     frac); report.fullLobeFraction=mean(full);
end

function tGrid=violet_phase_grid(gamma,q)
    NL=numel(gamma)-1; tGrid=nan(NL,numel(q)); th=violet_theta(gamma);
    for n=1:NL
        g0=gamma(n); g1=gamma(n+1); target=th(n)+q*(th(n+1)-th(n)); t=g0+(g1-g0)*q; t(1)=g0; t(end)=g1;
```

```
403         for it=1:12
404             cand=t-(violet_theta(t)-target)./violet_theta_prime(t); cand=min(max(cand,g0),g1); t(2:end-1)=
        cand(2:end-1);
405         end
406         t(1)=g0; t(end)=g1; tGrid(n,:)=t;
407     end
408 end
409
410 function y=violet_theta(t)
411     t=double(t); y=.5*t.*log(t/(2*pi))-.5*t-pi/8+1./(48*t)+7./(5760*t.^3)+31./(80640*t.^5)+127./(430080*t
        .^7)+511./(1216512*t.^9);
412 end
413
414 function y=violet_theta_prime(t)
415     t=double(t); y=.5*log(t/(2*pi))-1./(48*t.^2)-21./(5760*t.^4)-155./(80640*t.^6)-889./(430080*t.^8)
        -4599./(1216512*t.^10);
416 end
417
418 function Z=transverse_matrix(beta,t,Ncut,M,chunk)
419     beta=double(beta(:)); t=double(t(:).'); nb=numel(beta); nt=numel(t); Z=complex(zeros(nb,nt));
420     k=(1:Ncut-1).'; logk=log(k); A=exp(-beta*logk.'); logN=log(Ncut); Bernoulli=bernoulli_even(M);
421     for first=1:chunk:nt
422         last=min(nt,first+chunk-1); tt=t(first:last); B=exp(-1i*logk*tt); Zb=A*B; S=beta+1i*tt;
423         Zb=Zb+exp((1-S)*logN)./(S-1)+.5*exp(-S*logN);
424         for r=1:M
425             P=ones(size(S)); for j=0:(2*r-2), P=P.*(S+j); end
426             Zb=Zb+(Bernoulli(r)/factorial(2*r)).*P.*exp(-(S+(2*r-1))*logN);
427         end
428         Z(:,first:last)=Zb;
429     end
430 end
431
432 function B=bernoulli_even(M)
433     allB=[1/6;-1/30;1/42;-1/30;5/66;-691/2730;7/6;-3617/510;43867/798;-174611/330];
434     if M>numel(allB), error('Increase Bernoulli table.'); end; B=allB(1:M);
435 end
436
437 function w=trapezoid_weights(q)
438     q=q(:); h=q(2)-q(1); if max(abs(diff(q)-h))>1e-12, error('Uniform q grid required.'); end
439     w=h*ones(numel(q),1); w(1)=h/2; w(end)=h/2;
440 end
441
442 function out=row_median_finite(X)
443     out=nan(size(X,1),1); for i=1:size(X,1), out(i)=median_finite(X(i,:)); end
444 end
445
446 function m=median_finite(x)
447     x=x(isfinite(x)); if isempty(x), m=NaN; else, m=median(x); end
448 end
449
450 function rho=spearman_manual(x,y)
451     x=x(:); y=y(:); good=isfinite(x)&isfinite(y); x=x(good); y=y(good); if numel(x)<3, rho=NaN; return;
        end
452     rx=average_ranks(x); ry=average_ranks(y); rx=rx-mean(rx); ry=ry-mean(ry); den=sqrt(sum(rx.^2)*sum(ry
        .^2));
453     if den<=realmin, rho=NaN; else, rho=sum(rx.*ry)/den; end
454 end
455
456 function ranks=average_ranks(x)
457     [xs,order]=sort(x); N=numel(x); sr=zeros(N,1); i=1;
458     while i<=N
459         j=i; while j<N && xs(j+1)==xs(i), j=j+1; end; sr(i:j)=(i+j)/2; i=j+1;
460     end
461     ranks=zeros(N,1); ranks(order)=sr;
462 end
463
464 function save_figure_png(fig,path)
465     try, exportgraphics(fig,path,'Resolution',180); catch, set(fig,'PaperPositionMode','auto'); print(fig,
        path,'-dpng','-r180'); end
466 end
```

Listing 1. Complete MATLAB source for the five-quotient rigidity audit.

Appendix B. MATLAB: dense differential projective-flow audit

```
%% VIOLET_TRANSVERSE_PROJECTIVE_FLOW_AUDIT.m
% Pure MATLAB. Differential Transverse/Violet projective-flow audit.
%
% The fixed Violet lobe boundaries define q -> t_n(q).  The script evaluates
% W_{beta,n}, its exact transverse beta derivative (within the chosen
% Euler--Maclaurin representation), the normalized projective defect M,
% its analytic beta derivative M_beta, and the intrinsic projective speed P.
% No beta-dependent zeros are introduced or searched.

clear; clc; close all;
rng(20260825,'twister');

SCRIPT_DIR=fileparts(mfilename('fullpath'));
ROOT=fileparts(SCRIPT_DIR);

cfg=struct;
cfg.ZeroFile=fullfile(ROOT,'data','zeross.bin');
cfg.NLobes=240;
cfg.Nq=121;
cfg.NBetaLog=90;
cfg.NBetaLinear=181;
cfg.DeltaMin=1e-5;
cfg.EMOrder=8;
cfg.InitialCutoff=512;
cfg.MaxCutoff=4096;
cfg.TransverseAuditTol=2e-9;
cfg.TChunk=700;
cfg.SplineAnchorLeft=.10;
cfg.SplineAnchorRight=.90;
cfg.DerivativeAbsTol=2e-9;
cfg.DerivativeRelTol=2e-6;
cfg.ProjectiveSpeedTol=5e-10;
cfg.FDAuditBetaCount=15;
cfg.FDAuditLobeCount=30;
cfg.OutputDir=fullfile(ROOT,'outputs','projective_flow');
if ~exist(cfg.OutputDir,'dir'), mkdir(cfg.OutputDir); end

quotientNames={'HERMITE_JET1';'ENDPOINT_VALUES';'HERMITE_JET2';'MINIMAL_BOUNDARY_SPLINE';'ORTHOGONAL_CUBIC'};
NQ=numel(quotientNames);

fprintf('\nVIOLET TRANSVERSE PROJECTIVE FLOW AUDIT\n');
fprintf('------------------------------------------------------------\n');
fprintf('Violet lobes             : %d\n',cfg.NLobes);
fprintf('q samples/lobe           : %d\n',cfg.Nq);
fprintf('quotient families        : %d\n',NQ);

%% Fixed Violet boundaries
fid=fopen(cfg.ZeroFile,'rb'); if fid<0, error('Cannot open %s.',cfg.ZeroFile); end
gamma=fread(fid,cfg.NLobes+1,'double'); fclose(fid); gamma=gamma(:);
if numel(gamma)<cfg.NLobes+1, error('Not enough Violet lobe boundaries.'); end

%% Fixed phase projection
q=linspace(0,1,cfg.Nq); wq=trapezoid_weights(q); tGrid=violet_phase_grid(gamma,q); Tall=reshape(tGrid.',1,[]);
fprintf('\nVIOLET PHASE PROJECTION\n');
fprintf('------------------------------------------------------------\n');
fprintf('left endpoint error  = %.3e\n',max(abs(tGrid(:,1)-gamma(1:end-1))));
fprintf('right endpoint error = %.3e\n',max(abs(tGrid(:,end)-gamma(2:end))));

%% Dense beta grid
deltaLog=logspace(log10(cfg.DeltaMin),log10(.1),cfg.NBetaLog);
betaLinear=linspace(.5,1,cfg.NBetaLinear);
```

```
beta=unique([.5 .5+deltaLog betaLinear]).'; beta=beta(beta>=.5 & beta<=1); beta(1)=.5; beta(end)=1;
NB=numel(beta); NL=cfg.NLobes;
fprintf('\nDENSE TRANSVERSE DOMAIN\n');
fprintf('------------------------------------------------------------\n');
fprintf('beta samples          = %d\n',NB);
fprintf('Violet beta           = %.12f\n',beta(1));
fprintf('outer Transverse beta    = %.12f\n',beta(end));
fprintf('first positive delta  = %.3e\n',beta(2)-.5);

%% Spline anchors
[~,iL]=min(abs(q-cfg.SplineAnchorLeft)); [~,iR]=min(abs(q-cfg.SplineAnchorRight));
splineAnchorIndex=unique([1 iL iR cfg.Nq]); if numel(splineAnchorIndex)~=4, error('Spline anchors
    collapsed.'); end

%% Value and derivative audit
fprintf('\nTRANSVERSE / TRANSVERSE DERIVATIVE AUDIT\n');
fprintf('------------------------------------------------------------\n');
betaAudit=[.5;.50001;.5001;.51;.75;1]; pickT=round(linspace(1,numel(Tall),19)); tAudit=Tall(pickT); Ncut=
    cfg.InitialCutoff;
while true
    [Wa,dWa]=transverse_matrix_with_beta_derivative(betaAudit,tAudit,Ncut,cfg.EMOrder,cfg.TChunk);
    [Wb,dWb]=transverse_matrix_with_beta_derivative(betaAudit,tAudit,2*Ncut,cfg.EMOrder,cfg.TChunk);
    valueError=max(abs(Wa(:)-Wb(:))./max(1,abs(Wb(:))));
    derivativeError=max(abs(dWa(:)-dWb(:))./max(1,abs(dWb(:))));
    fprintf('cutoff %5d vs %5d : W %.3e   W_beta %.3e\n',Ncut,2*Ncut,valueError,derivativeError);
    if max(valueError,derivativeError)<=cfg.TransverseAuditTol, break; end
    Ncut=2*Ncut; if 2*Ncut>cfg.MaxCutoff, error('Transverse derivative audit failed.'); end
end
fprintf('accepted cutoff = %d\n',Ncut);

%% Complete differential Transverse
fprintf('\nBUILDING DIFFERENTIAL TRANSVERSE\n'); tic;
[Transverse,TransverseBeta]=transverse_matrix_with_beta_derivative(beta,Tall,Ncut,cfg.EMOrder,cfg.TChunk);
fprintf('elapsed = %.2f s\n',toc);

WVraw=reshape(Transverse(1,:),cfg.Nq,NL).';
fprintf('\nVIOLET BOUNDARY AUDIT\n');
fprintf('------------------------------------------------------------\n');
fprintf('max residual    = %.3e\n',max(abs([WVraw(:,1);WVraw(:,end)])));
fprintf('median residual = %.3e\n',median(abs([WVraw(:,1);WVraw(:,end)])));
WVraw(:,1)=0; WVraw(:,end)=0;

%% Storage
MByFamily=cell(NQ,1); MdByFamily=cell(NQ,1); PByFamily=cell(NQ,1);
FullPositiveDerivative=false(NL,NQ); StrictPositiveFraction=nan(NL,NQ); MinDerivativeByLobe=nan(NL,NQ);
Summary=table('Size',[NQ 9], ...
    'VariableTypes',{'string','double','double','double','double','double','double','double','double'}, ...

    'VariableNames',{'quotient','derivativeSampleFractionNonnegative','
     lobesDerivativeNonnegativeEverywhere', ...
    'lobeFractionDerivativeNonnegativeEverywhere','medianStrictPositiveFraction','
     minimumObservedDerivative', ...
    'minimumProjectiveSpeed','fdMedianRelativeError','fdMaximumRelativeError'});

%% Differential quotient loop
for jq=1:NQ
    qName=quotientNames{jq};
    fprintf('\nDIFFERENTIAL QUOTIENT %d / %d : %s\n',jq,NQ,qName);
    fprintf('------------------------------------------------------------\n');
    rhoV=apply_quotient(WVraw,q,wq,qName,splineAnchorIndex);
    [rV,~,goodV]=normalize_rows(rhoV,wq);
    fprintf('usable Violet lobes = %d / %d\n',nnz(goodV),NL);

    M=nan(NB,NL); Md=nan(NB,NL); P=nan(NB,NL);
    for ib=1:NB
        W=reshape(Transverse(ib,:),cfg.Nq,NL).';
        Wbeta=reshape(TransverseBeta(ib,:),cfg.Nq,NL).';
        if ib==1, W(:,1)=0; W(:,end)=0; end
        rho=apply_quotient(W,q,wq,qName,splineAnchorIndex);
        rhoBeta=apply_quotient(Wbeta,q,wq,qName,splineAnchorIndex);
```

```
        [r,normR,goodR]=normalize_rows(rho,wq); good=goodV & goodR;

        c=sum(conj(rV).*r.*wq.',2); align2=min(max(real(abs(c).^2),0),1); m=1-align2;
        a=sum(conj(rho).*rhoBeta.*wq.',2);
        rBeta=complex(nan(size(rho)));
        rBeta(good,:)=rhoBeta(good,:)./normR(good)-rho(good,:).*(real(a(good))./normR(good).^3);
        cBeta=sum(conj(rV).*rBeta.*wq.',2);
        md=-2*real(conj(c).*cBeta);
        rhoBetaNorm2=sum(abs(rhoBeta).^2.*wq.',2);
        p=rhoBetaNorm2./normR.^2-abs(a).^2./normR.^4;
        tiny=p<0 & p>=-cfg.ProjectiveSpeedTol; p(tiny)=0;
        m(~good)=NaN; md(~good)=NaN; p(~good)=NaN; if ib==1, m(good)=0; end
        M(ib,:)=m.'; Md(ib,:)=md.'; P(ib,:)=p.';
    end
    MByFamily{jq}=M; MdByFamily{jq}=Md; PByFamily{jq}=P;

    tol=cfg.DerivativeAbsTol+cfg.DerivativeRelTol.*max(abs(Md),1e-12);
    signPass=Md>=-tol; signStrict=Md>tol; finiteInterior=isfinite(Md(2:end,:));
    for n=1:NL
        good=finiteInterior(:,n); if ~any(good), continue; end
        FullPositiveDerivative(n,jq)=all(signPass(2:end,n) | ~good);
        StrictPositiveFraction(n,jq)=mean(signStrict(2:end,n) & good)/mean(good);
        MinDerivativeByLobe(n,jq)=min(Md(2:end,n),[],'omitnan');
    end
    vals=Md(2:end,:); finite=isfinite(vals); localTol=cfg.DerivativeAbsTol+cfg.DerivativeRelTol.*abs(vals);

    neg=finite & vals < -localTol; nNegative=nnz(neg); nTotal=nnz(finite); fractionNonnegative=1-nNegative
     /max(nTotal,1);

    finiteP=isfinite(P); minP=min(P(finiteP)); NnegativeP=nnz(P(finiteP)<-cfg.ProjectiveSpeedTol);

    %% Finite-difference audit of analytic M_beta
    betaIdx=unique(round(linspace(3,NB-2,cfg.FDAuditBetaCount)));
    lobeIdx=unique(round(linspace(1,NL,cfg.FDAuditLobeCount)));
    fdErrors=[];
    for ii=1:numel(betaIdx)
        ib=betaIdx(ii); hm=beta(ib)-beta(ib-1); hp=beta(ib+1)-beta(ib);
        for jj=1:numel(lobeIdx)
            n=lobeIdx(jj); y=[M(ib-1,n) M(ib,n) M(ib+1,n)]; if ~all(isfinite(y)), continue; end
            fd=finite_difference_nonuniform(y(1),y(2),y(3),hm,hp); ex=Md(ib,n);
            fdErrors(end+1,1)=abs(fd-ex)/max([abs(fd),abs(ex),1e-10]); %#ok<SAGROW>
        end
    end
    fdMedian=median(fdErrors); fdMax=max(fdErrors);

    fprintf('\nM_beta SIGN\n');
    fprintf('------------------------------------------------------------\n');
    fprintf('nonnegative derivative samples = %d / %d = %.9f\n',nTotal-nNegative,nTotal,
     fractionNonnegative);
    fprintf('lobes M_beta >= 0 for all beta = %d / %d\n',nnz(FullPositiveDerivative(:,jq)),NL);
    fprintf('median strict-positive fraction/lobe = %.9f\n',median(StrictPositiveFraction(:,jq),'omitnan')
     );
    fprintf('worst lobe minimum M_beta = %.12e\n',min(MinDerivativeByLobe(:,jq),[],'omitnan'));
    fprintf('\nPROJECTIVE SPEED P\n');
    fprintf('------------------------------------------------------------\n');
    fprintf('minimum P           = %.12e\n',minP);
    fprintf('invalid negative P  = %d\n',NnegativeP);
    fprintf('\nANALYTIC DERIVATIVE vs FINITE DIFFERENCE\n');
    fprintf('------------------------------------------------------------\n');
    fprintf('median relative error = %.6e\n',fdMedian);
    fprintf('maximum relative error = %.6e\n',fdMax);

    Summary.quotient(jq)=string(qName);
    Summary.derivativeSampleFractionNonnegative(jq)=fractionNonnegative;
    Summary.lobesDerivativeNonnegativeEverywhere(jq)=nnz(FullPositiveDerivative(:,jq));
    Summary.lobeFractionDerivativeNonnegativeEverywhere(jq)=mean(FullPositiveDerivative(:,jq));
    Summary.medianStrictPositiveFraction(jq)=median(StrictPositiveFraction(:,jq),'omitnan');
    Summary.minimumObservedDerivative(jq)=min(MinDerivativeByLobe(:,jq),[],'omitnan');
    Summary.minimumProjectiveSpeed(jq)=minP;
    Summary.fdMedianRelativeError(jq)=fdMedian;
```

```
    Summary.fdMaximumRelativeError(jq)=fdMax;
end

%% Cross-quotient diagnosis
allQ=all(FullPositiveDerivative,2); NallQ=nnz(allQ);
fprintf('\nCROSS-QUOTIENT DIFFERENTIAL RIGIDITY\n');
fprintf('------------------------------------------------------------\n');
fprintf('lobes with M_beta >= 0 for EVERY beta and EVERY Q: %d / %d = %.9f\n',NallQ,NL,NallQ/NL);
for jq=1:NQ, fprintf('%-28s : %3d / %3d\n',quotientNames{jq},nnz(FullPositiveDerivative(:,jq)),NL); end

fprintf('\nLOBE FAILURES\n');
fprintf('------------------------------------------------------------\n');
for jq=1:NQ
    bad=find(~FullPositiveDerivative(:,jq)); fprintf('%-28s : ',quotientNames{jq});
    if isempty(bad), fprintf('none'); else, fprintf('%d ',bad); end; fprintf('\n');
end

%% Tables
Tlobe=table((1:NL).',allQ,sum(FullPositiveDerivative,2), ...
    'VariableNames',{'violetLobe','nonnegativeForEveryQ','numberOfQPassed'});
for jq=1:NQ
    a=matlab.lang.makeValidName(['pass_' quotientNames{jq}]);
    b=matlab.lang.makeValidName(['minMd_' quotientNames{jq}]);
    c=matlab.lang.makeValidName(['strictFraction_' quotientNames{jq}]);
    Tlobe.(a)=FullPositiveDerivative(:,jq); Tlobe.(b)=MinDerivativeByLobe(:,jq); Tlobe.(c)=
     StrictPositiveFraction(:,jq);
end
writetable(Tlobe,fullfile(cfg.OutputDir,'LOBE_DIFFERENTIAL_SIGN_AUDIT.csv'));
writetable(Summary,fullfile(cfg.OutputDir,'PROJECTIVE_FLOW_SUMMARY.csv'));

%% Figures
f=figure('Name','Differential Transverse monotonicity','NumberTitle','off');
for jq=1:NQ, plot(beta,row_median_finite(MdByFamily{jq}),'LineWidth',1.4,'DisplayName',quotientNames{jq});
      hold on; end
yline(0,'--'); xlabel('\beta'); ylabel('median M_\beta'); title('Differential sign of Transverse/Violet
     projective deformation');
legend('Location','best','Interpreter','none'); grid on; save_figure_png(f,fullfile(cfg.OutputDir,'
     fig01_projective_derivative_Mbeta.png'));

f=figure('Name','Projective speed','NumberTitle','off');
for jq=1:NQ, semilogy(beta,max(row_median_finite(PByFamily{jq}),realmin),'LineWidth',1.4,'DisplayName',
     quotientNames{jq}); hold on; end
xlabel('\beta'); ylabel('median P'); title('Intrinsic projective speed of the Transverse fibres');
legend('Location','best','Interpreter','none'); grid on; save_figure_png(f,fullfile(cfg.OutputDir,'
     fig02_projective_speed_P.png'));

lobeShow=40;
f=figure('Name','Example differential Violet lobe','NumberTitle','off');
for jq=1:NQ, plot(beta,MdByFamily{jq}(:,lobeShow),'LineWidth',1.3,'DisplayName',quotientNames{jq}); hold
     on; end
yline(0,'--'); xlabel('\beta'); ylabel(sprintf('M''_{%d}(\beta)',lobeShow));
title(sprintf('Differential Transverse deformation of Violet lobe %d',lobeShow));
legend('Location','best','Interpreter','none'); grid on; save_figure_png(f,fullfile(cfg.OutputDir,'
     fig03_lobe40_Mbeta.png'));

save(fullfile(cfg.OutputDir,'VIOLET_TRANSVERSE_PROJECTIVE_FLOW_COMPLETE.mat'), ...
    'cfg','gamma','q','wq','tGrid','beta','quotientNames','MByFamily','MdByFamily','PByFamily', ...
    'FullPositiveDerivative','StrictPositiveFraction','MinDerivativeByLobe','Summary','-v7.3');

fprintf('\nFINAL VIOLET TRANSVERSE PROJECTIVE FLOW DIAGNOSIS\n');
fprintf('------------------------------------------------------------\n');
disp(Summary);
fprintf('lobes satisfying M_beta >= 0 for EVERY beta and EVERY quotient: %d / %d = %.9f\n',NallQ,NL,NallQ/
     NL);
fprintf('AUDIT COMPLETE\n');

%% Local functions
function rho=apply_quotient(W,q,wq,qName,splineAnchorIndex)
    switch upper(qName)
        case 'HERMITE_JET1', rho=quotient_hermite_jet1(W,q);
```

```
254         case 'ENDPOINT_VALUES', rho=quotient_endpoint_values(W,q);
255         case 'HERMITE_JET2', rho=quotient_hermite_jet2(W,q);
256         case 'MINIMAL_BOUNDARY_SPLINE', rho=quotient_boundary_spline(W,q,splineAnchorIndex);
257         case 'ORTHOGONAL_CUBIC', rho=quotient_orthogonal_cubic(W,q,wq);
258         otherwise, error('Unknown quotient %s.',qName);
259     end
260 end
261
262 function rho=quotient_hermite_jet1(W,q)
263     [d0,d1]=endpoint_first_derivatives(W,q); y0=W(:,1); y1=W(:,end); x=q(:).';
264     h00=2*x.^3-3*x.^2+1; h10=x.^3-2*x.^2+x; h01=-2*x.^3+3*x.^2; h11=x.^3-x.^2;
265     H=y0.*h00+d0.*h10+y1.*h01+d1.*h11; rho=W-H; rho(:,1)=0; rho(:,end)=0;
266 end
267 function rho=quotient_endpoint_values(W,q)
268     x=q(:).'; H=W(:,1).*(1-x)+W(:,end).*x; rho=W-H; rho(:,1)=0; rho(:,end)=0;
269 end
270 function rho=quotient_hermite_jet2(W,q)
271     [d0,d1]=endpoint_first_derivatives(W,q); [dd0,dd1]=endpoint_second_derivatives(W,q); y0=W(:,1); y1=W(:,
        end);
272     A=[1 0 0 0 0 0;0 1 0 0 0 0;0 0 2 0 0 0;1 1 1 1 1 1;0 1 2 3 4 5;0 0 2 6 12 20];
273     C=A\[y0.';d0.';dd0.';y1.';d1.';dd1.']; x=q(:); V=[ones(size(x)) x x.^2 x.^3 x.^4 x.^5]; H=(V*C).'; rho
        =W-H; rho(:,1)=0; rho(:,end)=0;
274 end
275 function rho=quotient_boundary_spline(W,q,idx)
276     H=complex(zeros(size(W))); qa=q(idx); for n=1:size(W,1), H(n,:)=spline(qa,W(n,idx),q); end; rho=W-H;
277 end
278 function rho=quotient_orthogonal_cubic(W,q,wq)
279     x=q(:); B=[ones(size(x)) x x.^2 x.^3]; G=B.'*(B.*wq); RHS=B.'*(W.'.*wq); C=G\RHS; rho=W-(B*C).';
280 end
281 function [d0,d1]=endpoint_first_derivatives(W,q)
282     h=q(2)-q(1); d0=(-25*W(:,1)+48*W(:,2)-36*W(:,3)+16*W(:,4)-3*W(:,5))/(12*h);
283     d1=(25*W(:,end)-48*W(:,end-1)+36*W(:,end-2)-16*W(:,end-3)+3*W(:,end-4))/(12*h);
284 end
285 function [dd0,dd1]=endpoint_second_derivatives(W,q)
286     h=q(2)-q(1); dd0=(35*W(:,1)-104*W(:,2)+114*W(:,3)-56*W(:,4)+11*W(:,5))/(12*h^2);
287     dd1=(35*W(:,end)-104*W(:,end-1)+114*W(:,end-2)-56*W(:,end-3)+11*W(:,end-4))/(12*h^2);
288 end
289 function [R,nrm,good]=normalize_rows(X,wq)
290     nrm=sqrt(sum(abs(X).^2.*wq.',2)); good=isfinite(nrm)&nrm>1e-12; R=complex(nan(size(X))); R(good,:)=X(
        good,:)./nrm(good);
291 end
292 function d=finite_difference_nonuniform(ym,y0,yp,hm,hp)
293     d=-(hp/(hm*(hm+hp)))*ym+(hp-hm)/(hm*hp)*y0+(hm/(hp*(hm+hp)))*yp;
294 end
295 function tGrid=violet_phase_grid(gamma,q)
296     NL=numel(gamma)-1; tGrid=nan(NL,numel(q)); th=violet_theta(gamma);
297     for n=1:NL
298         g0=gamma(n); g1=gamma(n+1); target=th(n)+q*(th(n+1)-th(n)); t=g0+(g1-g0)*q; t(1)=g0; t(end)=g1;
299         for it=1:12
300             cand=t-(violet_theta(t)-target)./violet_theta_prime(t); cand=min(max(cand,g0),g1); t(2:end-1)=
        cand(2:end-1);
301         end
302         t(1)=g0; t(end)=g1; tGrid(n,:)=t;
303     end
304 end
305 function y=violet_theta(t)
306     t=double(t); y=.5*t.*log(t/(2*pi))-.5*t-pi/8+1./(48*t)+7./(5760*t.^3)+31./(80640*t.^5)+127./(430080*t
        .^7)+511./(1216512*t.^9);
307 end
308 function y=violet_theta_prime(t)
309     t=double(t); y=.5*log(t/(2*pi))-1./(48*t.^2)-21./(5760*t.^4)-155./(80640*t.^6)-889./(430080*t.^8)
        -4599./(1216512*t.^10);
310 end
311
312 function [Z,dZ]=transverse_matrix_with_beta_derivative(beta,t,Ncut,M,chunk)
313     beta=double(beta(:)); t=double(t(:).'); nb=numel(beta); nt=numel(t); Z=complex(zeros(nb,nt)); dZ=
        complex(zeros(nb,nt));
314     k=(1:Ncut-1).'; logk=log(k); A=exp(-beta*logk.'); Ad=-A.*logk.'; logN=log(Ncut); Bernoulli=
        bernoulli_even(M);
315     for first=1:chunk:nt
```

```
        last=min(nt,first+chunk-1); tt=t(first:last); B=exp(-1i*logk*tt); Zb=A*B; dZb=Ad*B; S=beta+1i*tt;
        T1=exp((1-S)*logN)./(S-1); dT1=T1.*(-logN-1./(S-1)); Zb=Zb+T1; dZb=dZb+dT1;
        T2=.5*exp(-S*logN); dT2=-logN*T2; Zb=Zb+T2; dZb=dZb+dT2;
        for r=1:M
            Product=ones(size(S)); LogDerivative=zeros(size(S));
            for j=0:(2*r-2), Product=Product.*(S+j); LogDerivative=LogDerivative+1./(S+j); end
            corr=(Bernoulli(r)/factorial(2*r)).*Product.*exp(-(S+(2*r-1))*logN);
            dcorr=corr.*(LogDerivative-logN); Zb=Zb+corr; dZb=dZb+dcorr;
        end
        Z(:,first:last)=Zb; dZ(:,first:last)=dZb;
    end
end
function B=bernoulli_even(M)
    allB=[1/6;-1/30;1/42;-1/30;5/66;-691/2730;7/6;-3617/510;43867/798;-174611/330];
    if M>numel(allB), error('Increase Bernoulli table.'); end; B=allB(1:M);
end
function w=trapezoid_weights(q)
    q=q(:); h=q(2)-q(1); if max(abs(diff(q)-h))>1e-12, error('Uniform q grid required.'); end; w=h*ones(
     numel(q),1); w(1)=h/2; w(end)=h/2;
end
function out=row_median_finite(X)
    out=nan(size(X,1),1); for i=1:size(X,1), x=X(i,:); x=x(isfinite(x)); if ~isempty(x), out(i)=median(x);
      end; end
end
function save_figure_png(fig,path)
    try, exportgraphics(fig,path,'Resolution',180); catch, set(fig,'PaperPositionMode','auto'); print(fig,
     path,'-dpng','-r180'); end
end
```

Listing 2. Complete MATLAB source for the dense projective derivative and speed audit.

Appendix C. MATLAB: pointwise curvature, reflection, and Pólya diagnostics

```
%% RAINBOW_REFLECTION_POLYA_AUDIT.m
% Pure MATLAB.
%
% Orthogonal-cubic follow-up audit for the transverse Violet family.
% It performs four checks that are logically distinct:
%   1) direct pointwise mixed-curvature C(alpha,beta) scans for Q_perp;
%   2) reflection-conjugation decomposition u = X + Y with a canonical
%      lobe-wise phase chi chosen at beta=1/2;
%   3) modal phase-locking and corrected adjacent Chebyshev determinants
%         Delta_m = p_m p'_{m+1} - p_{m+1} p'_m;
%   4) finite Polya generating-polynomial root diagnostics.
%
% The supplied ordinates are used only as fixed beta=1/2 lobe boundaries.
% No beta-dependent zeros are introduced or searched.

clear; clc; close all;
rng(20260825,'twister');

SCRIPT_DIR=fileparts(mfilename('fullpath'));
ROOT=fileparts(SCRIPT_DIR);

cfg=struct;
cfg.ZeroFile=fullfile(ROOT,'data','zeross.bin');
cfg.Lobes=[2 8 25];
cfg.Nq=241;
cfg.BetaCount=[11 21 11]; % endpoints later omitted from C summary
cfg.EMOrder=8;
cfg.InitialCutoff=512;
cfg.MaxCutoff=4096;
cfg.EvaluatorTol=2e-9;
cfg.TChunk=700;
cfg.MaxMode=18;
cfg.RootImagTol=1e-8;
cfg.RootRealTol=1e-10;
```

```
cfg.SignTol=1e-10;
cfg.OutputDir=fullfile(ROOT,'outputs','reflection_polya');
if ~exist(cfg.OutputDir,'dir'), mkdir(cfg.OutputDir); end

fprintf('\nRAINBOW REFLECTION / POLYA AUDIT\n');
fprintf('================================================================\n');
fprintf('lobes                  : '); fprintf('%d ',cfg.Lobes); fprintf('\n');
fprintf('q samples              : %d\n',cfg.Nq);
fprintf('maximum modal degree   : %d\n',cfg.MaxMode);

%% Fixed lobe boundaries
maxLobe=max(cfg.Lobes);
fid=fopen(cfg.ZeroFile,'rb');
if fid<0, error('Cannot open %s.',cfg.ZeroFile); end
gamma=fread(fid,maxLobe+1,'double'); fclose(fid); gamma=gamma(:);
if numel(gamma)<maxLobe+1, error('Not enough fixed lobe boundaries.'); end

q=linspace(0,1,cfg.Nq).';
wq=trapezoid_weights(q);

%% Weighted orthonormal polynomial basis.
% Start from shifted Legendre columns and re-orthonormalize under the exact
% discrete quadrature used everywhere else in this audit.  The first four
% columns span P_3 exactly on the grid.
Phi=shifted_legendre_basis(q,cfg.MaxMode);
A=sqrt(wq).*Phi;
[Qm,~]=qr(A,0);
Psi=Qm./sqrt(wq);
for j=1:size(Psi,2)
    if real(sum(Psi(:,j).*Phi(:,j).*wq))<0, Psi(:,j)=-Psi(:,j); end
end
Gram=Psi'*(Psi.*wq);
fprintf('weighted modal Gram error = %.3e\n',max(abs(Gram-eye(size(Gram))),[],'all'));

%% Evaluator cutoff audit on all selected lobes.
tAudit=[];
for il=1:numel(cfg.Lobes)
    n=cfg.Lobes(il);
    tg=violet_phase_grid(gamma(n:n+1),q.');
    tAudit=[tAudit tg(1,round(linspace(1,cfg.Nq,9)))]; %#ok<AGROW>
end
betaAudit=[.5;.50001;.51;.75;1];
Ncut=cfg.InitialCutoff;
while true
    [Wa,dWa]=transverse_matrix_with_beta_derivative(betaAudit,tAudit,Ncut,cfg.EMOrder,cfg.TChunk);
    [Wb,dWb]=transverse_matrix_with_beta_derivative(betaAudit,tAudit,2*Ncut,cfg.EMOrder,cfg.TChunk);
    eW=max(abs(Wa(:)-Wb(:))./max(1,abs(Wb(:))));
    eD=max(abs(dWa(:)-dWb(:))./max(1,abs(dWb(:))));
    fprintf('cutoff %d vs %d : W %.3e   W_beta %.3e\n',Ncut,2*Ncut,eW,eD);
    if max(eW,eD)<=cfg.EvaluatorTol, break; end
    Ncut=2*Ncut;
    if 2*Ncut>cfg.MaxCutoff, error('Evaluator cutoff audit failed.'); end
end
fprintf('accepted cutoff = %d\n',Ncut);

CurvatureSummary=table('Size',[numel(cfg.Lobes) 6], ...
    'VariableTypes',{'double','double','double','double','double','double'}, ...
    'VariableNames',{'lobe','betaGridCount','interiorPairCount','minimumC','maximumC','negativeCount'});
ReflectionRows=[];
ChebyshevRows=[];
RootRows=[];

for il=1:numel(cfg.Lobes)
    n=cfg.Lobes(il);
    NB=cfg.BetaCount(il);
    beta=linspace(.5,1,NB).';
    tGrid=violet_phase_grid(gamma(n:n+1),q.');
    t=tGrid(1,:);

    [W,Wb]=transverse_matrix_with_beta_derivative(beta,t,Ncut,cfg.EMOrder,cfg.TChunk);
```

```
    U=quotient_orthogonal_cubic(W,q,wq);
    Ub=quotient_orthogonal_cubic(Wb,q,wq);

    %% 1) Direct pointwise mixed curvature.
    C=nan(NB,NB);
    for ia=1:NB
        ua=U(ia,:); dua=Ub(ia,:);
        for ib=1:NB
            ub=U(ib,:); dub=Ub(ib,:);
            K=weighted_ip(ua,ub,wq);
            if abs(K)<1e-14, continue; end
            C(ia,ib)=real(weighted_ip(dua,dub,wq)/K - ...
                weighted_ip(dua,ub,wq)*weighted_ip(ua,dub,wq)/(K*K));
        end
    end
    interior=C(2:end-1,2:end-1); vals=interior(isfinite(interior));
    CurvatureSummary.lobe(il)=n;
    CurvatureSummary.betaGridCount(il)=NB;
    CurvatureSummary.interiorPairCount(il)=numel(vals);
    CurvatureSummary.minimumC(il)=min(vals);
    CurvatureSummary.maximumC(il)=max(vals);
    CurvatureSummary.negativeCount(il)=nnz(vals<0);

    fprintf('\nLOBE %d : POINTWISE MIXED CURVATURE\n',n);
    fprintf('------------------------------------------------------------\n');
    fprintf('beta grid            = %d\n',NB);
    fprintf('interior pairs       = %d\n',numel(vals));
    fprintf('min C                = %.12e\n',min(vals));
    fprintf('max C                = %.12e\n',max(vals));
    fprintf('negative C           = %d\n',nnz(vals<0));

    %% 2) Canonical reflection gauge at Violet.
    uV=U(1,:);
    JOuV=conj(fliplr(uV));
    z=weighted_ip(uV,JOuV,wq);
    if abs(z)>0, exp2ichi=exp(-1i*angle(z)); else, exp2ichi=1; end
    chi=.5*angle(exp2ichi);

    X=complex(zeros(size(U))); Y=complex(zeros(size(U)));
    Xb=complex(zeros(size(U))); Yb=complex(zeros(size(U)));
    for ib=1:NB
        Ju=exp2ichi*conj(fliplr(U(ib,:)));
        Jub=exp2ichi*conj(fliplr(Ub(ib,:)));
        X(ib,:)=(U(ib,:)+Ju)/2;
        Y(ib,:)=(U(ib,:)-Ju)/2;
        Xb(ib,:)=(Ub(ib,:)+Jub)/2;
        Yb(ib,:)=(Ub(ib,:)-Jub)/2;
    end

    normU=row_weighted_norm(U,wq); normY=row_weighted_norm(Y,wq);
    normUb=row_weighted_norm(Ub,wq); normYb=row_weighted_norm(Yb,wq);

    % Exact coefficient a(q)=theta'(t(q))/Omega and its reflection defect.
    Omega=violet_theta(gamma(n+1))-violet_theta(gamma(n));
    aq=violet_theta_prime(t)./Omega;
    ap=(aq+fliplr(aq))/2; am=(aq-fliplr(aq))/2;
    asymAbs=max(abs(am));
    asymRel=asymAbs/max(min(abs(ap)),realmin);

    % Modal coefficients, including low modes for diagnostic purposes.
    coefU=modal_coefficients(U,Psi,wq);
    coefUb=modal_coefficients(Ub,Psi,wq);
    coefX=modal_coefficients(X,Psi,wq);
    coefY=modal_coefficients(Y,Psi,wq);

    m=(0:cfg.MaxMode).';
    phaseX=coefX .* exp(-1i*chi) .* ((-1i).^m);       % should be real
    phaseY=coefY .* exp(-1i*chi) .* ((-1i).^(m+1));   % should be real
    high=5:(cfg.MaxMode+1); % MATLAB indices for modes >=4
    lockX=max(abs(imag(phaseX(high,:))),[],'all')/max(max(abs(phaseX(high,:)),[],'all'),realmin);
```

```
    lockY=max(abs(imag(phaseY(high,:))),[],'all')/max(max(abs(phaseY(high,:)),[],'all'),realmin);

    fprintf('\nLOBE %d : REFLECTION DECOMPOSITION\n',n);
    fprintf('------------------------------------------------------------\n');
    fprintf('chi                  = %.12e\n',chi);
    fprintf('max ||Y||/||u||      = %.12e\n',max(normY./max(normU,realmin)));
    fprintf('max ||Y_beta||/||u_beta|| = %.12e\n',max(normYb./max(normUb,realmin)));
    fprintf('max |a_-|            = %.12e\n',asymAbs);
    fprintf('max |a_-|/min|a_+|   = %.12e\n',asymRel);
    fprintf('X modal phase defect  = %.3e\n',lockX);
    fprintf('Y modal phase defect  = %.3e\n',lockY);

    for ib=1:NB
        ReflectionRows=[ReflectionRows; n beta(ib) chi normY(ib)/max(normU(ib),realmin) ...
            normYb(ib)/max(normUb(ib),realmin) asymAbs asymRel]; %#ok<AGROW>
    end

    %% 3) Corrected adjacent Chebyshev determinants for full high-mode u.
    coeff=coefU(high,:); coeffb=coefUb(high,:);
    p=abs(coeff).^2;
    pp=2*real(conj(coeff).*coeffb);
    for ib=1:NB
        for r=1:size(p,1)-1
            Delta=p(r,ib)*pp(r+1,ib)-p(r+1,ib)*pp(r,ib);
            ChebyshevRows=[ChebyshevRows; n beta(ib) (r+3) Delta]; %#ok<AGROW>
        end
    end

    dvals=ChebyshevRows(ChebyshevRows(:,1)==n,4);
    fprintf('\nLOBE %d : ADJACENT CHEBYSHEV SIGN\n',n);
    fprintf('------------------------------------------------------------\n');
    fprintf('Delta_m samples       = %d\n',numel(dvals));
    fprintf('nonnegative fraction  = %.9f\n',mean(dvals>=-cfg.SignTol));
    fprintf('minimum Delta_m       = %.12e\n',min(dvals));

    %% 4) Polya generating-polynomial root diagnostic.
    % G(z)=sum_{r=0}^{R} p_{r+4} z^r.  This is a sufficient-route audit,
    % not an assumption and not needed for the direct C calculation.
    for ib=1:NB
        cpoly=p(:,ib).';
        cpoly=cpoly/max(cpoly);
        rr=roots(fliplr(cpoly));
        nReal=nnz(abs(imag(rr))<=cfg.RootImagTol);
        nNonPos=nnz(abs(imag(rr))<=cfg.RootImagTol & real(rr)<=cfg.RootRealTol);
        maxImag=max(abs(imag(rr)));
        maxReal=max(real(rr));
        RootRows=[RootRows; n beta(ib) numel(rr) nReal nNonPos maxImag maxReal]; %#ok<AGROW>
    end
end

Tref=array2table(ReflectionRows,'VariableNames', ...
    {'lobe','beta','chi','relativeY','relativeYBeta','maxAbsAMinus','relativeAMinus'});
Tcheb=array2table(ChebyshevRows,'VariableNames',{'lobe','beta','mode','DeltaAdjacent'});
Troot=array2table(RootRows,'VariableNames', ...
    {'lobe','beta','rootCount','realRootCount','nonpositiveRealRootCount','maxAbsImag','maxReal'});

writetable(CurvatureSummary,fullfile(cfg.OutputDir,'POINTWISE_CURVATURE_SUMMARY.csv'));
writetable(Tref,fullfile(cfg.OutputDir,'REFLECTION_DEFECT.csv'));
writetable(Tcheb,fullfile(cfg.OutputDir,'ADJACENT_CHEBYSHEV_DETERMINANTS.csv'));
writetable(Troot,fullfile(cfg.OutputDir,'POLYA_ROOT_DIAGNOSTIC.csv'));

fprintf('\n============================================================\n');
fprintf('POINTWISE CURVATURE SUMMARY\n');
fprintf('============================================================\n');
disp(CurvatureSummary);
fprintf('Output directory: %s\n',cfg.OutputDir);
fprintf('AUDIT COMPLETE\n');

%% Local functions
function z=weighted_ip(x,y,w)
```

```
    z=sum(conj(x(:)).*y(:).*w(:));
end

function n=row_weighted_norm(X,w)
    n=sqrt(sum(abs(X).^2.*w.',2));
end

function C=modal_coefficients(U,Psi,w)
    C=Psi'*(U.'.*w); % modes x beta
end

function rho=quotient_orthogonal_cubic(W,q,wq)
    B=[ones(numel(q),1) q q.^2 q.^3];
    G=B'*(B.*wq);
    RHS=B'*(W.'.*wq);
    coeff=G\RHS;
    rho=W-(B*coeff).';
end

function Phi=shifted_legendre_basis(q,M)
    x=2*q-1;
    Phi=zeros(numel(q),M+1);
    P0=ones(size(x));
    Phi(:,1)=P0;
    if M==0, return; end
    P1=x; Phi(:,2)=sqrt(3)*P1;
    for m=1:M-1
        P2=((2*m+1)*x.*P1-m*P0)/(m+1);
        Phi(:,m+2)=sqrt(2*(m+1)+1)*P2;
        P0=P1; P1=P2;
    end
end

function tGrid=violet_phase_grid(gamma,q)
    NL=numel(gamma)-1; tGrid=nan(NL,numel(q)); th=violet_theta(gamma);
    for n=1:NL
        g0=gamma(n); g1=gamma(n+1); target=th(n)+q*(th(n+1)-th(n));
        t=g0+(g1-g0)*q; t(1)=g0; t(end)=g1;
        for it=1:12
            cand=t-(violet_theta(t)-target)./violet_theta_prime(t);
            cand=min(max(cand,g0),g1); t(2:end-1)=cand(2:end-1);
        end
        t(1)=g0; t(end)=g1; tGrid(n,:)=t;
    end
end

function y=violet_theta(t)
    t=double(t);
    y=.5*t.*log(t/(2*pi))-.5*t-pi/8+1./(48*t)+7./(5760*t.^3)+ ...
      31./(80640*t.^5)+127./(430080*t.^7)+511./(1216512*t.^9);
end

function y=violet_theta_prime(t)
    t=double(t);
    y=.5*log(t/(2*pi))-1./(48*t.^2)-21./(5760*t.^4)- ...
      155./(80640*t.^6)-889./(430080*t.^8)-4599./(1216512*t.^10);
end

function [Z,dZ]=transverse_matrix_with_beta_derivative(beta,t,Ncut,M,chunk)
    beta=double(beta(:)); t=double(t(:).'); nb=numel(beta); nt=numel(t);
    Z=complex(zeros(nb,nt)); dZ=complex(zeros(nb,nt));
    k=(1:Ncut-1).'; logk=log(k); A=exp(-beta*logk.'); Ad=-A.*logk.';
    logN=log(Ncut); Bernoulli=bernoulli_even(M);
    for first=1:chunk:nt
        last=min(nt,first+chunk-1); tt=t(first:last); B=exp(-1i*logk*tt);
        Zb=A*B; dZb=Ad*B; S=beta+1i*tt;
        T1=exp((1-S)*logN)./(S-1); dT1=T1.*(-logN-1./(S-1));
        Zb=Zb+T1; dZb=dZb+dT1;
        T2=.5*exp(-S*logN); dT2=-logN*T2; Zb=Zb+T2; dZb=dZb+dT2;
        for r=1:M
```

```
            Product=ones(size(S)); LogDerivative=zeros(size(S));
            for j=0:(2*r-2)
                Product=Product.*(S+j); LogDerivative=LogDerivative+1./(S+j);
            end
            corr=(Bernoulli(r)/factorial(2*r)).*Product.*exp(-(S+(2*r-1))*logN);
            dcorr=corr.*(LogDerivative-logN); Zb=Zb+corr; dZb=dZb+dcorr;
        end
        Z(:,first:last)=Zb; dZ(:,first:last)=dZb;
    end
end

function B=bernoulli_even(M)
    allB=[1/6;-1/30;1/42;-1/30;5/66;-691/2730;7/6;-3617/510;43867/798;-174611/330];
    if M>numel(allB), error('Increase Bernoulli table.'); end
    B=allB(1:M);
end

function w=trapezoid_weights(q)
    q=q(:); h=q(2)-q(1);
    if max(abs(diff(q)-h))>1e-12, error('Uniform q grid required.'); end
    w=h*ones(numel(q),1); w(1)=h/2; w(end)=h/2;
end
```

Listing 3. MATLAB source for the direct mixed-curvature reconstruction, reflection-conjugation defect, corrected adjacent Chebyshev determinants, and finite Pólya-root diagnostics.

Bio Logic Crop Science S.L.